

PaperPass[免费版]AIGC检测报告

简明打印版

AIGC总体疑似度(高+中+轻): 34.8%

AIGC总体疑似度(高+中+轻): 23.3% (加权计算)

高度疑似AIGC占全文比: 1.8%
中度疑似AIGC占全文比: 28.02%
轻度疑似AIGC占全文比: 4.98%
不予检测文字占比: 15.2%

检测版本: 免费版(仅检测中文)

报告编号: 2YQT69D79B1C12749

论文题目: 基于大数据技术的个性化音乐推荐系统的设计与实现 (V8)

论文作者: 卢伟军

论文字数: 55461

段落个数: 1350

句子个数: 2148

片段个数: 330

提交时间: 2026-4-9 20:27:08

查询真伪: <https://www.paperpass.com/check>

疑似度分布图:



正文中片段的颜色表示不同的疑似度范围

- 红色: AIGC生成疑似度在70%以上 (高度疑似)
- 橙色: AIGC生成疑似度在60%~70% (中度疑似)
- 紫色: AIGC生成疑似度在50%~60% (轻度疑似)
- 黑色: AIGC生成疑似度在50%以下
- 灰色: 过短片段、标题和英文等不予检测的文字

注: AIGC检测可有效识别文本是否部分或全部由AI模型生成, 检测结果与论文质量无关、仅表示论文中内容片段存在AI生成可能性的概率。

加权计算公式: (片段1疑似度 + 片段2疑似度 + ... + 片段n疑似度) / n
片段疑似度范围0.0~1.0, 合格片段按照0计算, 不计算不予检测文字

广州华立学院

本科生毕业论文（设计）

题目：基于大数据技术的个性化音乐推荐系
统设计与实现

学	院	计算机工程学院
专	业	数据科学与大数据技术
班	级	Z1 大数据
学	号	5124440106796
姓	名	卢伟军
指导教师		王波

2026 年 4 月

原创性声明

本人郑重声明：在广州华立学院攻读学士学位期间，所提交的毕业论文（设计）《基于大数据技术的个性化音乐推荐系统设计与实现》，是本人在导师指导下独立进行研究工作所取得的成果。对本文的研究工作做出重要贡献的个人和集体，均已在文中以明确方式注明，其它未注明部分不包含他人已发表或撰写过的研究成果，不存在购买、由他人代写、剽窃和伪造数据等作假行为。

本人愿为此声明承担法律责任。

作者签名：卢伟军.

日期： 2026 年 4 月 20 日

摘 要

随着数字音乐流媒体平台的用户规模及曲库容量的持续增长，用户的信息过载现象已经日益显著，如何从海量候选曲目中优化用户潜在偏好，是个性化音乐服务的关键技术挑战。音乐消费行为的隐性、用户兴趣的迁移以及长尾曲目的交互稀疏，使推荐任务建模变得困难。本文基于 KKBOX 公开音乐数据集，设计并实现了以 MusicWeb 与 MusicMode 为核心的个性化音乐推荐系统，并探索大数据处理与多模型协同精排在音乐推荐场景中的工程集成。系统特征工程层面构建多层的用户行为统计、歌曲属性与协同过滤嵌入特征管道，折外目标编码的方式避免高基数类别特征训练后标签泄露的问题。推荐算法由多通道召回—粗排—精排集成—重排四层管道构建，多通道召回层并行产生约 600 首候选曲目；梯度提升树模型粗排，将候选集降至 300 首；DeepFM 和 BST 行为序列 Transformer 对 top-300 候选进行特征建模，经 SLSQP 约束加权融合双模型预测加权；重排部分引入最大边际相关性算法，在推荐质量和列表多样性之间进行权衡，最终给每个用户推荐 50 首左右的推荐结果。工程层面结合外部音乐数据接口，通过隐式反馈闭环机制保证推荐结果不断迭代改进。离线与在线双轨评估结果表明，四层推荐管道的排序精度和候选覆盖情况均较好，在有历史偏好和冷启动两种用户情况下均达到了基本可用的推荐效果。

关键词：大数据技术；个性化推荐；深度学习；集成学习；协同过滤；特征工程

Abstract

With the growth of user size and music library available on digital music streaming platforms, it is becoming more and more important to match users' potential preferences from massive candidate tracks to facilitate personalized music services. The implicit nature of the music consumption, the user's drift and the interaction density of long tail tracks further complicate the recommendation modeling. To cope with these challenges, this thesis designs and implements a personalized music recommendation system from the KKBOX public music dataset with a front-end interaction platform (MusicWeb) and a recommendation engine (MusicMode) to explore the engineering integration of big data processing and multi-model collaborative fine-ranking in music recommendation problems. In the feature engineering stage, we build a multi-dimensional feature pipeline based on user behavior statistics, song attributes, and collaborative filtering embeddings. An out-of-fold target encoding is employed to prevent label leak from high-cardinality categorical features in training. Based on the recommendation algorithm, a four-layer model of multi-channel recall, coarse ranking, fine-rating ensemble, and reranking is utilized. Recall layer generates approximately 600 candidate tracks in parallel; gradient boosting tree model compresses candidates to 300; DeepFM and Behavior Sequence Transformer (BST) model deep features for the top-300 candidates and fuse their predictions using SLSQP-optimized weighted combination; and the reranking stage considers the Maximal Marginal Relevance algorithm between recommendation quality and diversity of the list and finally produces 50 recommendations per user. External music data interfaces at the engineering level, with an implicit feedback loop for continuous recommendation tuning. Offline and online dual-track evaluation results show that the proposed four-layered pipeline yields satisfactory ranking accuracy and candidate coverage. The system provides functionally effective recommendations for both rich historical preference and cold-start users with limited

interaction history.

Keywords: Big Data Technology, Personalized Recommendation, Deep Learning, Ensemble Learning, Collaborative Filtering, Feature Engineering

目 录

摘 要..... I

Abstract..... II

1 绪 论..... 1

1.1 研究背景与意义 1

1.2 国内外研究现状 1

1.2.1 国内研究现状 1

1.2.2 国外研究现状 2

1.3 论文研究内容 3

1.4 论文组织结构 4

2 系统开发技术与算法选型 6

2.1 大数据技术 6

2.1.1 PySpark 分布式计算框架..... 6

2.1.2 Redis 缓存技术 6

2.2 协同过滤与向量检索技术 7

2.2.1 协同过滤与 ALS 矩阵分解..... 7

2.2.2 向量近邻检索技术（FAISS） 8

2.3 梯度提升树与 LightGBM 原理 9

2.4 精排深度模型 11

2.4.1 DeepFM 模型原理 11

2.4.2 BST 行为序列 Transformer 原理 13

2.5 Stacking 集成学习框架 15

2.6 Java Web 核心技术栈 16

2.6.1 MVC 架构与 Servlet/JSP 技术..... 16

2.7 本章小结 17

3 系统需求分析..... 19

3.1	系统总体目标	19
3.2	功能性需求分析	19
3.2.1	用户端功能需求	19
3.2.2	管理端功能需求	21
3.2.3	推荐引擎功能需求	22
3.3	非功能性需求分析	23
3.3.1	性能需求	23
3.3.2	安全性需求	24
3.4	本章小结	25
4	系统设计	26
4.1	引言	26
4.2	系统总体架构设计	26
4.2.2	MusicMode 推荐引擎模块架构	28
4.2.3	双模块交互与部署设计	29
4.3	推荐算法流水线设计	30
4.3.1	召回层设计	30
4.3.2	粗排层与精排层设计	32
4.3.3	集成层设计	33
4.3.4	重排层设计	33
4.4	数据库设计	35
4.4.1	表的概念结构设计	35
4.4.2	表的逻辑结构设计	36
4.5	前端交互设计	36
4.5.1	页面总体结构设计	36
4.5.2	核心功能页面设计	37
4.5.3	推荐交互与行为反馈设计	38

4.6	本章小结	39
5	系统实现	41
5.1	引言	41
5.2	开发环境与工具配置	41
5.3	数据处理与特征工程实现	43
5.3.1	数据来源与 ETL 处理	43
5.3.2	数据清洗	45
5.3.3	特征工程	46
5.3.4	数据集时序切分	50
5.4	MusicMode 推荐算法实现	50
5.4.1	ALS 协同过滤召回实现	51
5.4.2	FAISS 向量检索召回实现	51
5.4.3	热度兜底召回实现	51
5.4.4	LightGBM 粗排实现	52
5.4.5	DeepFM 深度精排实现	53
5.4.6	BST 行为序列 Transformer 精排实现	54
5.4.7	Stacking 集成与重排输出的实现	55
5.5	MusicWeb 前端实现	56
5.5.1	MVC 分层架构搭建	56
5.5.2	用户认证与账户管理实现	57
5.5.3	个性化推荐展示	59
5.5.4	音乐搜索与播放收藏实现	60
5.5.5	歌单管理功能实现	61
5.5.6	管理员后台功能实现	62
5.5.7	隐式反馈采集实现	65
5.6	本章小结	65

6	系统测试与评估	67
6.1	引言	67
6.2	评估指标与实验设置	67
6.2.1	评估指标说明	67
6.2.2	实验设置	69
6.3	消融实验结果分析	70
6.4	各模型 AUC 对比实验.....	72
6.5	本章小结	73
	结 论.....	75
	参考文献	77
	致 谢	79
	附 录.....	81

1 绪 论

1.1 研究背景与意义

随着数字音乐流媒体平台规模的不断扩张，以 Spotify、网易云音乐为代表的主流平台曲库数量都达到了亿首量级，用户通过人工榜单或者搜索关键词来发现音乐的效率极低，信息过载问题日益突出。通过对用户过往行为的建模，将庞大候选曲目压缩为潜在偏好值的少量结果是各大平台维持用户活跃度的关键技术基础^[1]。音乐推荐面临的技术问题：用户评价多为播放时间、跳曲等隐式行为，正负样本标注噪声较大；音乐消费具有极强的情境依赖性，静态偏好模型无法捕捉到用户兴趣的动态变化；长尾曲目交互记录极为稀疏，协同过滤方法在冷启动场景下性能退化严重^[2]。

上述问题需要推荐系统在算法设计与工程实现两个维度都进行系统化处理，本文将基于 KKBOX 公开数据集设计并实现 MusicWeb 与 MusicMode 构成的双模块个性化音乐推荐系统，探索大数据处理与多模型异构精排在离线音乐场景下的工程集成，为中小型在线音乐平台提供参考。

1.2 国内外研究现状

1.2.1 国内研究现状

AI 70% 在工程化实践方向，赵吉[1]基于分布式计算框架构建了以矩阵分解协同过滤为核心的大规模音乐推荐系统，验证了分布式计算路径的工程可行性，但未引入深度学习精排层，对特征交叉信息的利用较为有限；杨率[2]将深度神经网络应用于个性化实时音乐推荐，探索了用户行为序列的动态偏好建模，然而其系统缺乏完整的前端交互平台，推荐结果的用户反馈闭环有待完善。上述两项工作初步论证了大数据技术在音乐推荐场景中的适用性，但均未形成召回、精排与重排的多层流水线，本文在此基础上引入多模型集成精排管道，以期在推荐精度与系统完整性上有所推进。

AI 70% 在语义增强与情感特征方向，李津[3]引入知识图谱丰富物品侧语义表示，对缓解

稀疏场景下的冷启动问题有一定效果，但图卷积的计算复杂度较高，难以适配百万量级曲库的实时推理需求；冯亚寒[4]提出了融合小波散射变换与多尺度卷积网络的音乐多情感分类方法，在情感特征提取上具有一定新颖性，但该方法聚焦于内容侧标签预测，未与用户行为数据联合建模，难以直接用于个性化推荐；吴亚迪和陈平华[5]设计了融合用户长短期偏好与情感注意力的推荐模型，在个性化偏好建模方面有所改进，但两者均依赖显式情感标注，在以隐式行为反馈为主的大规模数据集上的适用性有待进一步验证。

AI 67%

在综述与前沿方法方向，王慧心和童向荣[6]梳理了融合知识图谱的推荐系统研究进展，指出语义增强对于缓解数据稀疏性具有较好效果，但计算开销制约了其在实时推荐中的落地；梁锋等[7]对基于联邦学习的推荐系统隐私保护机制进行了综述，为在不共享原始数据条件下的协同建模提供了参考思路；郑楠等[8]对基于深度学习的群组推荐方法进行了分类比较，总结了不同群体偏好聚合策略的适用边界；闫猛等[9]提出了基于自监督学习的序列推荐算法，在减少对人工标注数据依赖方面具有一定借鉴价值；龚雪鸾等[10]对点击率预测方法进行了系统综述，梳理了从浅层模型到深度模型的演进路径，为本文精排层的模型选型提供了参考依据。上述综述类研究揭示了现有方法在计算效率、隐式反馈利用及多阶特征交叉等方面的共性不足，本文在精排层采用 DeepFM 与 BST 的组合，正是对这一不足的针对性回应。

AI 66%

综合来看，现有国内研究在单一模型创新方面已有较多积累，但针对多模型集成精排管道的完整工程实现、前端交互平台与大数据推荐引擎的异构集成，以及特征工程阶段标签泄漏的系统性防控，仍缺乏系统性的公开论述，本文将就上述方面展开针对性研究与实践。

1.2.2 国外研究现状

AI 69%

在音乐推荐专项研究方向，Liu 等[11]提出情感驱动的个性化音乐推荐框架，通过对歌词与音频的情感分析构建用户情绪—偏好映射；Wan 等[12]设计了多视图增强图注意力网络用于基于会话的音乐推荐，在多个公开数据集上取得了较好表现；Li 等

[13]提出注意力自编码器用于内容感知型推荐，在交互稀疏场景下的性能优于纯协同过滤基线；Mao 等[14]提出轻量化音乐分类推荐网络，探索了分类任务与推荐任务联合建模的可行性；Paul 等[15]对协同过滤、内容过滤与混合推荐三类范式进行了冷启动与稀疏场景下的适用边界比较；Tran 等[19]将 Transformer 架构与认知科学中的记忆激活理论相结合，专门针对音乐收听场景中频繁出现的重复播放行为进行建模，为序列化音乐推荐中的用户行为分析提供了新的研究视角。上述研究在情感特征、内容建模与序列行为分析方面各有贡献，但多数工作聚焦于单一模型的改进，对多模型协同精排管道的完整工程集成探索相对不足。

AI 70%

在深度精排与行为序列建模方向，Chen 等[20]将 Transformer 架构引入推荐系统精排层，提出行为序列 BST，通过多头自注意力机制对用户历史行为序列中的时序依赖关系进行建模，相较于注意力池化类方法在点击率预测任务上展现出一定的性能优势。该工作表明 Transformer 在用户行为序列建模中具有较大潜力，为本文将行为序列 Transformer 引入音乐精排场景提供了重要的方法论参考。

AI 70%

在大规模工程方向，Niu 等[16]基于分布式计算框架实现了大规模协同过滤音乐推荐，验证了分区策略对矩阵分解训练效率的提升作用；Johnson 等[17]提出的高性能向量近邻检索库已广泛应用于工业界推荐系统的召回层；Wang 等[18]通过神经因果图协同过滤对交互数据中的混淆因素进行显式建模，对缓解观察偏差对推荐结果的影响具有一定参考价值。上述工程方向的研究多聚焦于单一技术组件的优化，对多组件端到端集成方案的系统性论述相对有限。

AI 69%

综合来看，现有国内外研究在单模型创新与特定场景优化方面已有较多积累，但针对多模型异构精排管道的完整工程实现、前端平台与大数据推荐引擎的异构集成以及特征工程中标签泄漏的系统性防控，仍缺乏全面的公开论述，本文将围绕上述方面展开研究与实践。

1.3 论文研究内容

本文将重点解决以下四个核心问题：

（1）大规模音乐数据的高效治理与无泄漏特征工程

AI 62%

如何将基于 KKBOX 数据集构建全量 ETL 导入与特征工程管道，并采用 OOF (Out-of-Fold, 折外预测) 目标编码方案系统性消除训练集与验证集之间的标签泄漏风险，为粗排层与精排层各模型分别构建独立的多维特征集供模型训练使用。

（2）四层推荐管道构建与双模型 Stacking 集成

如何将 LightGBM、DeepFM 和 BST 共同置于四层推荐管道中，并实现三路召回通道汇聚约 600 首候选歌曲，然后 LightGBM 粗排将候选集压缩至 Top-300。并且使用 DeepFM 与 BST 对 Top-300 进行并行精排，通过 5 折交叉验证生成堆外预测(OOF)，以逻辑回归元学习器学习两者的融合权重，形成两阶段 Stacking 集成，输出 Top-150 精排结果，最终经 MMR 多样性重排产生 Top-50 个性化推荐列表。

（3）MusicWeb 音乐平台全功能工程实现

如何基于 Java Web 技术栈构建 MVC 分层架构的在线音乐平台，实现覆盖用户注册登录、个人信息管理、个性化推荐展示、音乐搜索、在线播放与收藏、歌单管理及管理员后台五大模块，并通过节流机制异步采集用户播放行为，经 C3P0 连接池高并发持久化写入，构建推荐反馈闭环。

（4）推荐系统离线评估体系构建

如何针对四层推荐管道与 Stacking 集成架构，设计合理的离线评估方案，在有效量化各管道模块实际增益的同时，横向比较各单体模型与集成框架的点击率判别能力，从而验证系统整体设计的合理性与有效性。

1.4 论文组织结构

本研究分为六章，附录部分有支撑性的材料，各章节内容结构安排如下所示。第一章以个性化音乐推荐学术背景为出发点，回顾国内外相关研究进展，确定本文的研究重点和实施策略；第二章系统梳理了大数据处理技术、推荐系统理论基础、深度学习推荐模型、集成学习框架以及 Web 开发技术的基本原理；第三章从功能性、非功能性角度对系统建立的目标进行了分析，并编写了详细需求说明书；第四章对双模块

整体架构、四层推荐算法体系以及数据库的设计策略做了详细分析；第五章展示了数据预处理、特征工程提取、算法实现、前端交互等具体操作过程；第六章通过消融实验与模型对比实验两组离线评估，对系统各模块的推荐性能进行量化验证和实验分析。

2 系统开发技术与算法选型

2.1 大数据技术

2.1.1 PySpark 分布式计算框架

Apache Spark 属于一款专门针对大规模数据处理所构建的内存计算引擎，在迭代式机器学习场景里具备明显优势，其运算速度比传统依靠磁盘进行批量处理的方式高得多。PySpark 是 Apache Spark 官方 Python 接口工具，用简化开发流程设计理念把分布式数据访问、SQL 查询解析和机器学习算法等功能模块集成为一个模块。Spark 核心组件 RDD（Resilient Distributed Dataset）有三个关键技术特点，分别是利用惰性求值方式达到全局最优全局规划，依靠血缘容错机制当节点失效时凭借转换历史来重塑消失的数据片段，而且能够自适应地改变并行度，使资源得到最大利用。在此基础上，Spark 加强了对结构化数据存储的支持能力，使用列式压缩，极大地提高了高维特征矩阵运算速度以及整体吞吐量表现。

本系统采用本地多核处理器环境，用 PySpark 做主要功能实现。针对 KKBOX 公开数据集里存在的海量交互记录，用 PySpark 做全方位的数据清洗以及特征工程工作，为其后分析打下基础。PySpark 把协同过滤算法（ALS 模型等）与离线训练结合起来，采用分布式计算的方式做用户行为、音乐项目的潜在因子离线训练，产生嵌入式向量模型。它们既可以支持协同过滤模块的运转，也可以成为召回算法里的主要输入。封装高效分布式计算引擎，并且提供统一标准接口，PySpark 大大提高了大规模数据预处理效率和推荐系统开发速度，是 MusicMode 智能音乐推荐平台的重要底层计算单元。

2.1.2 Redis 缓存技术

Redis 作为主流键值存储方案，用内存作为主要数据存储介质，具有很强的并发性能，可以达到亚毫秒级响应延迟来满足大量访问需求。其内置键过期功能可以自动收回无效或过期数据条目，从而大大提高资源利用效率，能够有效改善数据管理性

以及稳定性。基于单线程事件驱动架构设计思想，Redis 通过取消多线程环境下锁的竞争以及频繁的上下文切换，在高负载时仍可取得很好性能及稳定吞吐量。

本系统前端服务模块的在线推荐功能对于响应时延有较高技术要求。其主要业务流程包含推荐结果展示、用户偏好状态提取和个性化筛选条件匹配等环节，这些操作比较频繁并且常常伴随高并发请求，对外部系统非常敏感。当核心请求直接指向传统的基于关系型数据库时，在高并发时会出现明显查询效率降低和性能被过度占用的情况。由于 Redis 建立的内存级缓存有明显优势，即亚毫秒级读写速度、数据结构多样化和智能数据淘汰方式，所以可有效降低后端数据库压力，并且极大提高系统整体效率和稳定性。采用 Redis 做前置缓存层不仅可以提高资源利用率，还可以提升用户体验。

2.2 协同过滤与向量检索技术

2.2.1 协同过滤与 ALS 矩阵分解

协同过滤(Collaborative Filtering)是一种经典的推荐算法，它利用用户或历史行为来发现潜在的联系。根据不同的目标对象，主要分为基于用户 CF 和基于物品 CF 两种类型，前者通过用户偏好构造相似性矩阵来发现附近用户，后者则关注历史行为之间的相似性，向目标项给出个性化推荐。在数据量小、维度低的情况下，由于在工程上较易使用，且计算时间较少，因此物品 CF 比用户 CF 更为适用。面对数据稀疏问题时，传统 CF 模型经常会陷入严重偏差之中，超过 90%的缺失值会对预测精度造成较大影响，并且随着样本数量的增加，存储要求会变得越来越高的，同时计算效率也会随之降低，因此它很难适应目前大规模分布式环境下的即时响应需求。

AI 64%

矩阵分解方法将高维稀疏的用户-物品交互矩阵近似分解为用户因子矩阵与物品因子矩阵的乘积（即低秩近似），其中每个用户和每首歌曲分别被映射为一个 k 维稠密潜在因子向量，通过潜变量空间中的几何距离关系隐式捕捉用户偏好与物品特征之间的语义关联，从而消除原始矩阵因大量缺失值引起的预测不确定性，同时将存储与计算开销从原始矩阵规模大幅压缩。根据公式（2-1），对任意用户 u 和其对应喜好

物品 i ，利用两者潜在因子向量内积来预测评分值。

$$\hat{r}_{ui} = p_u^T q_i \quad (2-1)$$

式中 p_u^T ——用户 u 的 k 维潜在因子向量的转置（行向量）；

q_i ——物品 i 的 k 维潜在因子向量；内积值越大表明用户对该物品的潜在兴趣越强。

$\hat{r}_{ui}$ ——用户 u 对物品 i 的预测偏好得分。

音乐推荐领域的用户行为数据（播放频次、时长等）属于隐式反馈，其最突出的特征是只能体现用户积极的偏好倾向，不能很好地体现用户消极偏好信息。为克服上述固有缺陷，因此因此交替最小二乘（Alternating Least Squares, ALS）算法加入置信度加权法，用交互事件的数量来归一化观测样本权重，从而保持未观测样本的预测性，又不把缺失交互样本误认为是隐式否定。该算法核心优化目标可用公式（2-2）来描述。

$$L = \sum_{u,i} \text{conf}_{ui} (p_{ui} - p_u^T q_i)^2 + \lambda \left(\sum_u |p_u|^2 + \sum_i |q_i|^2 \right) \quad (2-2)$$

式中 p_{ui} ——二值化偏好指示变量，有交互记录时取 1，否则取 0；

conf_{ui} ——置信度权重，随原始交互计数增大而增大；

λ ——L2 正则化系数，用于抑制模型过拟合。

ALS 算法采用迭代更新两个因子矩阵的方式来分别优化另一个矩阵，将原来的非凸优化问题一步一步地转换成若干个局部最优凸子问题。每次迭代都可得到解析表达式的形式解，不但有较好的大规模并行执行能力，还有很强的分布式计算兼容性。

本文把大量隐式反馈数据作为对象，建立一个包含 229 万首候选歌曲和 737 万条交互记录的分析框架，对用户与音乐之间存在着的语义联系进行详细的探究。为提高算法效率，利用 ALS 做矩阵分解，很好地处理隐式反馈数据集，将置信度权重加入播放行为特征，利用高效交替迭代完成分布式计算。该技术路线可以加快因子矩阵学习速度，能够得到一个可用来做协同过滤推荐以及向量相似性检索的高精度嵌入模型。

2.2.2 向量近邻检索技术（FAISS）

AI 67%

作为大规模推荐系统的核心部分，向量近邻检索（Approximate Nearest Neighbor, ANN）为本系统提供坚实根基。Meta 推出的开源框架 FAISS 具有很好的性能指标，是创建高效召回模块推荐系统的主要手段，其采用基于 Inverted File 和 Product Quantization (IVFPQ) 的双层索引结构，在提高查询效率的同时有效控制了内存占用。通过粗粒度 IVF 分组机制将所有待检索对象分成多个子簇，针对用户兴趣特点只对相关性高的候选簇进行细致挑选，极大削减了全量数据扫描可能。在产品量化处理环节，使用子空间分解对高维向量做低维表示，从而在保持较高准确率和全面性的同时，降低了存储要求。

本系统从规模达到 229 万首、每个候选歌曲包含 160 维嵌入向量的候选音乐库里，精确地找到符合用户喜好目标的内容。如果使用传统逐条遍历、计算余弦相似度等方法，在每一次查询时都会进行几十到上百次复杂高维向量运算，无法满足实时推荐服务对于高效响应的要求。因此，在设计阶段采用 FAISS 框架来创建 IVFPQ 索引，以求在精度和性能之间达到一个较好的平衡并提高搜索效率，较好地解决大规模向量集存储与管理的问题，为系统正常工作奠定技术基础。

2.3 梯度提升树与 LightGBM 原理

AI 70%

梯度提升树（Gradient Boosting Decision Tree, GBDT）是一种以决策树为弱学习器的迭代集成方法，其核心理念是逐步逼近，即每一轮训练目标并非直接预测真实标签，而是专门拟合当前集成模型在所有样本上的预测残差（负梯度方向），逐步将整体误差压缩至最小。设经过第 T 轮迭代后集成模型的预测值为 $\hat{F}_T(x)$ ，其加法更新规则如式（2-3）所示：

$$\hat{F}_T(x) = \hat{F}_{T-1}(x) + \eta \cdot h_T(x) \tag{2-3}$$

式中 $\hat{F}_{T-1}(x)$ ——前 T-1 轮集成模型的预测值；

$h_T(x)$ ——第 T 轮新增的决策树弱学习器；

AI 64%

η ——学习率，控制每棵树对最终预测结果的贡献步长，取值越小模型泛化能力越强。

LightGBM 是微软开源的高效 GBDT 框架，针对大规模高维特征场景引入了三项关键工程优化：直方图算法将连续特征离散化为 K 个区间，节点分裂时仅需遍历 K 个候选切分点而非全部样本值，将单次分裂时间复杂度从 $O(n)$ 降至 $O(K)$ ；基于梯度的单边采样（GOSS）优先保留大梯度样本以减少计算量；互斥特征捆绑（EFB）将互斥稀疏特征合并，进一步降低特征维度，LightGBM 的算法原理如图 2-1 所示。

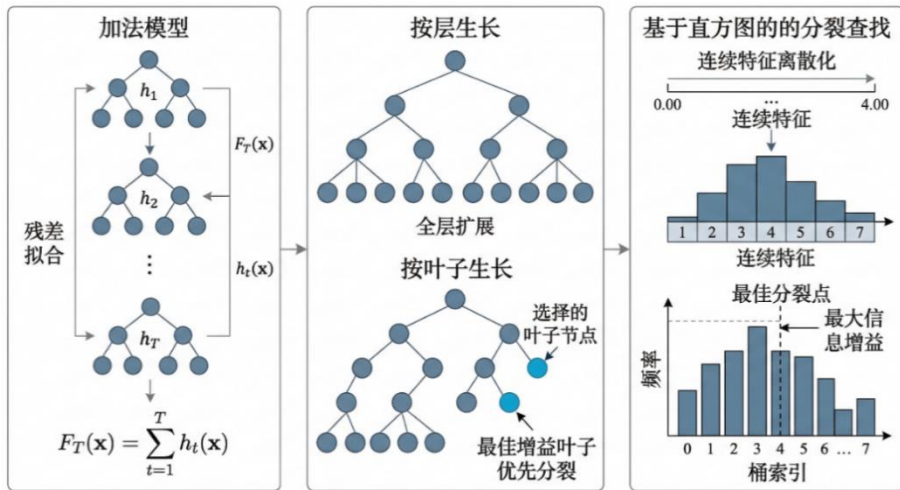


图2-1 LightGBM算法原理示意图

LightGBM 以分裂增益作为节点划分评判标准，各候选切分点增益计算如式（2-4）所示：

$$Gain = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma \quad (2-4)$$

式中 G_L 、 G_R ——分别为左、右子节点所有样本的一阶梯度之和；

H_L 、 H_R ——分别为左、右子节点所有样本的二阶梯度之和；

λ ——叶节点权重的 L2 正则化系数，惩罚过大的叶节点输出值；

γ ——引入新叶节点的最小收益阈值，增益低于该值时分裂被剪枝，起结构正则化作用。

LightGBM 使用叶子为基础的生长方式，在节点划分时先考虑增加的收益最大叶节点，在相同的叶节点数目限制下，大大降低模型训练过程中的预测偏差。

粗排阶段需在较短时间内对约 600 首召回候选进行初步评分，从中筛选出 300 首进入精排，且须有效建模用户侧、歌曲侧与上下文侧共 59 维混合特征中大量非线性

交叉关系。LightGBM 的直方图优化将节点分裂复杂度从 $O(n)$ 降至 $O(K)$ ，叶优先生长策略在相同叶节点数量约束下可获得更低的训练误差，配合增益阈值剪枝对模型复杂度进行结构性约束，使其在高维稀疏输入下兼具训练效率与泛化能力。上述特性与粗排阶段对吞吐效率和特征建模深度的双重要求高度契合，因此本系统将 LightGBM 作为粗排层的评分模型。

2.4 精排深度模型

2.4.1 DeepFM 模型原理

DeepFM 通过共享 Embedding 层将传统因子分解机（Factorization Machine, FM）与深度神经网络（DNN）整合于同一端到端框架中联合训练，无需人工特征工程即可同时捕获低阶与高阶特征交互，其模型结构示意图如图 2-2 所示。DeepFM 由 FM 组件与 Deep 组件并联构成，两部分共享同一 Embedding 层输出，分别承担不同阶次的特征交叉建模任务。

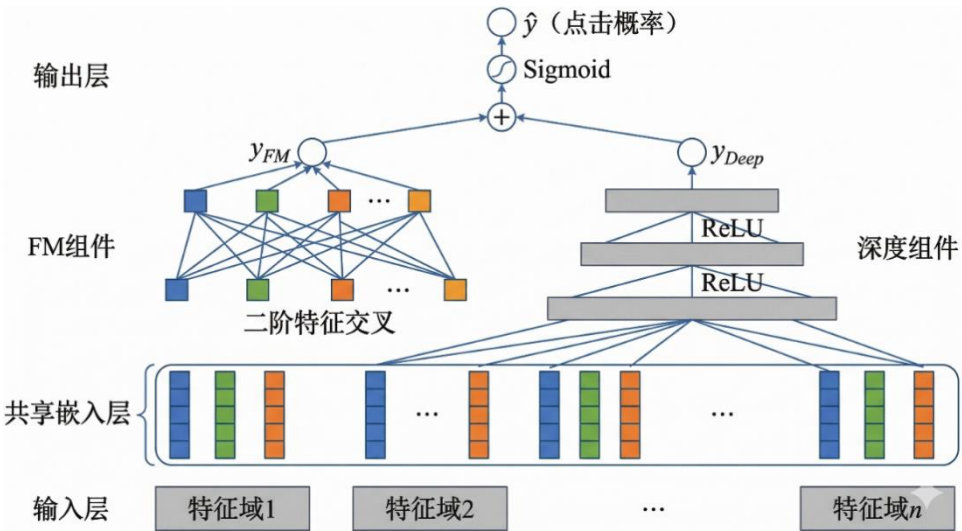


图2-2 DeepFM模型结构示意图

DeepFM 由 FM 组件与 Deep 组件并联构成，两部分共享同一 Embedding 层输出，分别承担不同阶次的特征交叉建模任务。

(1) FM 组件：

FM 组件负责捕捉一阶线性关系与二阶特征交叉。对于含 n 个特征域的输入，FM

组件的输出如式（2-5）所示：

$$y_{FM} = w_0 + \sum_{i=1}^n w_i x_i + \sum_{i=1}^n \sum_{j=i+1}^n \langle v_i, v_j \rangle x_i x_j \quad (2-5)$$

式中 w_0 ——全局偏置项；

w_i ——第 i 个特征的一阶线性权重；

v_i ——第 i 个特征对应的 K 维 Embedding 隐向量；

AI 69%

x_i 、 x_j ——第 i 、 j 个特征的输入值。

上式通过等价变换将二阶特征交叉的计算复杂度从 $O(n^2)$ 降至 $O(nK)$ ，同时以隐向量内积 $\langle v_i, v_j \rangle$ 衡量特征 i 与特征 j 的交叉强度，即使对于训练集中未曾共现的特征对，也能通过隐向量泛化估计其交叉权重。

（2）Deep 组件

Deep 组件为多层全连接神经网络，以各特征域 Embedding 向量的拼接结果作为输入，通过逐层非线性变换挖掘高阶特征交叉关系，第 1 层前向传播规则见式（2-6）所示：

$$a^{(l+1)} = \sigma(W^{(l)}a^{(l)} + b^{(l)}) \quad (2-6)$$

式中 $a^{(l)}$ ——第 l 层神经元输出向量；

$W^{(l)}$ ——第 l 层权重矩阵；

$b^{(l)}$ ——第 l 层偏置向量；

σ ——激活函数（本系统采用 ReLU）。

（3）联合预测输出

DeepFM 将 FM 组件与 Deep 组件输出相加后，经 Sigmoid 函数映射至 $[0,1]$ 区间，作为用户对候选歌曲的点击概率预测值，见式（2-7）。

$$\hat{y} = \sigma(y_{FM} + y_{Deep}) \quad (2-9)$$

式中 $\hat{y}$ ——用户对候选歌曲的点击概率预测值；

y_{FM} ——FM 组件的输出值；

y_{Deep} ——Deep 组件的输出值；

AI 60%

$\sigma(\cdot)$ ——Sigmoid 激活函数，将两组件输出之和压缩至 $(0,1)$ 区间，使结果可解释

为点击概率。

在特征提取模块(FM)和深层网络(Deep)共享同一个 Embedding 层一起参与参数优化时，对于分阶段训练容易陷入局部最优解的问题有比较明显的缓解作用。

在精排环节中，候选曲目特征体系由用户属性、音乐特性、用户歌曲互动等各个方面组成，关联关系是线性和非线性交织在一起的复杂关系。利用 DeepFM 架构设计把低阶、高阶特征结合起来用共同的空间维度来映射，再采取端到端联合训练方法促使二者深层次合作。由于本模型对于高阶特征组合具有较好适应性以及泛化性能，因此本系统将其作为推荐系统精排模块的重要评分工具之一。

2.4.2 BST 行为序列 Transformer 原理

AI 70%
传统深度推荐模型在提取用户兴趣表征时，通常对历史行为序列中所有交互物品的 Embedding 向量直接求平均，忽视了行为序列的时序依赖与上下文语义。行为序列 Transformer（Behavior Sequence Transformer, BST）将 Transformer 的自注意力机制引入行为序列建模，通过位置感知的多头自注意力层在全序列范围内对历史行为物品的相互依赖关系进行联合建模，使模型能够同时感知序列的局部兴趣模式与全局上下文语义。BST 整体结构如图 2-3 所示，对于含 N 条历史行为记录的用户，将各物品的 Embedding 向量与位置编码相加，得到每个位置的序列输入表示，再经 Transformer 层为每一位置生成融合全局上下文的输出表示。

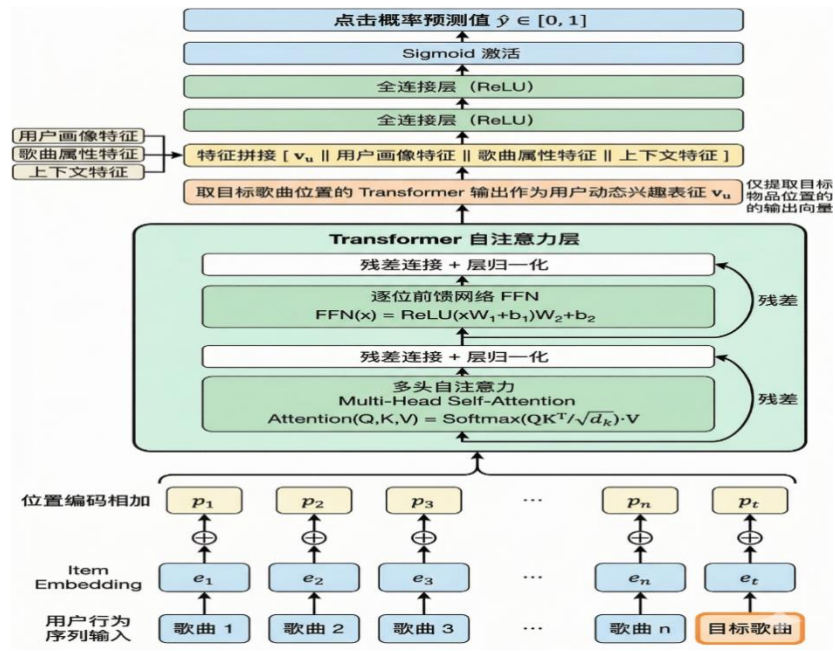


图2-3 BST模型结构示意图

以单头自注意力为例，其计算如式（2-8）所示：

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{(QK^T)}{\sqrt{(d_k)}}\right)V \quad (2-8)$$

式中 Q ——由序列输入经线性投影得到的查询矩阵；

K ——键矩阵；

V ——值矩阵；

AI 70%

d_k ——键向量维度，用作缩放因子以防止内积值过大导致梯度消失。

多头自注意力在 h 个不同子空间并行执行上述运算后拼接输出，再经前馈网络与残差连接进行处理，如式（2-9）所示：

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_j)W^O \quad (2-9)$$

式中 head_j ——第 j 个注意力头的输出，由 $\text{head}_j = \text{Attention}(QW_jQ, KW_jK, VW_jV)$ 计算得到；

W^O ——输出投影矩阵；

h ——注意力头数。

在本系统中，用户音乐偏好往往随收听序列演变而动态变化，近期播放行为对当

前兴趣的反映强于久远记录，行为间的顺序关系也携带着兴趣迁移语义信息。BST 通过位置编码感知行为时序位置，通过多头自注意力捕捉序列内部依赖关系，能够从用户历史播放序列中提取比均值池化更为精细的动态兴趣表征，较好满足精排阶段对序列感知能力的需求，因此选用 BST 作为本系统精排阶段的序列建模组件。

2.5 Stacking 集成学习框架

AI 70%

集成学习（Ensemble Learning）是一种通过组合多个学习器的预测结果来提升整体泛化能力的机器学习范式。其核心思想在于：单一模型受归纳偏置的限制，往往只能捕捉数据中特定维度的规律，而多个异质模型因结构差异产生的偏差相互独立，通过合理融合可有效降低整体预测误差。集成方法正是利用这种模型间的互补性，在不改变单个学习器结构的前提下，将多个模型的预测信息整合为更鲁棒的输出。

Stacking 是集成学习中的一种两阶段训练框架，其核心机制在于通过交叉验证生成元特征，再由元学习器在元特征空间中学习最优融合策略，其框架如图 2-4 所示。

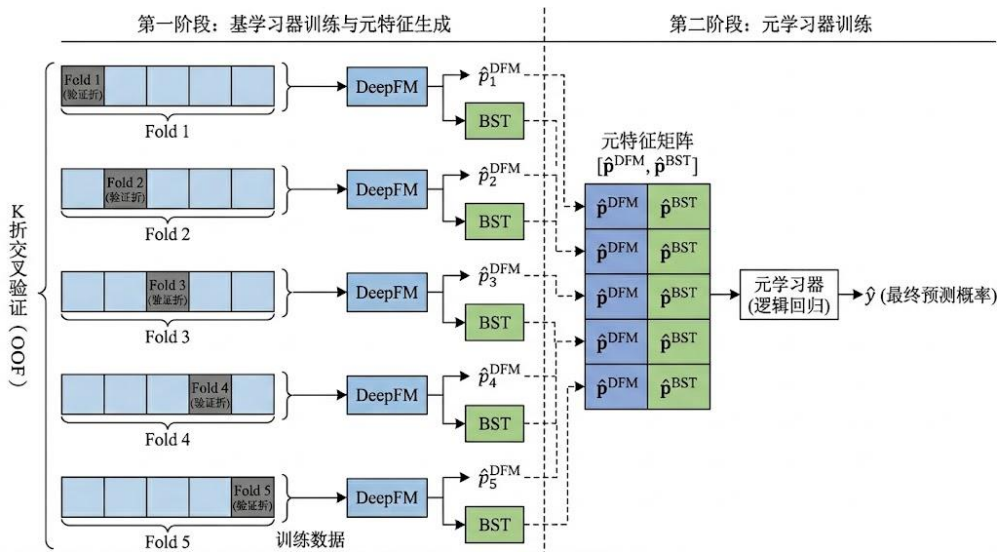


图2-4 Stacking两阶段训练框架示意图

AI 66%

第一阶段采用 K 折交叉验证，将训练集划分为 K 个互不重叠的子集，对第 k 折而言，基学习器在其余 K-1 折上完成训练，并对第 k 折样本进行预测，最终遍历所有折后得到覆盖全体训练样本的堆外预测（Out-of-Fold, OOF），可用公式（2-10）来描述：

$$\hat{p}_i = f_k(x_i), i \in \text{Fold}_k, k \in \{1, \dots, K\} \tag{2-10}$$

式中 k ——折编号， k 取 1 至 K 的整数；
 F ——在第 k 折外训练集上拟合的基学习器；
 x_i ——第 k 折中第 i 个样本的输入特征向量；
 $\hat{p}_i$ ——样本 x_i 对应的 OOF 元特征预测值。

第二阶段以全体训练样本的 OOF 预测值构成新的元特征矩阵，训练元学习器学习各基学习器输出到最终概率的映射关系，如公式（2-11）所示：

$$\hat{y} = \sigma!(w_1\hat{p}^{DFM} + w_2\hat{p}^{BST} + b) \tag{2-11}$$

式中 $\hat{p}^{DFM}$ ——DeepFM 模型在训练集上生成的 OOF 预测值；
 $\hat{p}^{BST}$ ——BST 模型在训练集上生成的 OOF 预测值；
 w_1 、 w_2 ——元学习器对应两个基学习器输出的权重参数；
 b ——元学习器偏置项；
 $\sigma(\cdot)$ ——Sigmoid 激活函数；
 $\hat{y}$ ——元学习器输出的最终融合概率。

本文以 DeepFM 与 BST 作为基学习器，以逻辑回归作为元学习器，构建两阶段 Stacking 集成框架用于推荐系统精排阶段。DeepFM 擅长捕捉用户与歌曲属性间的二阶特征交叉，BST 通过 Transformer 机制建模用户时序行为依赖，两者在特征表达上具有天然互补性。K 折交叉验证设置为 $K=5$ ，并依据样本全局播放时间戳进行时序切分，确保 OOF 预测过程不出现未来信息泄漏。该框架能够在保持基学习器独立性的同时，通过元学习器自适应挖掘两种模型预测结果间的互补关系，提升推荐系统对用户偏好的综合判断能力。

2.6 Java Web 核心技术栈

2.6.1 MVC 架构与 Servlet/JSP 技术

MVC 模式是根据职责分离原则将一个软件应用系统划分成三部分操作。模型层主要处理业务规则、数据管理，视图层主要是展示数据内容，控制器层则是处理用户

交互请求，并且各模块之间要相互配合。本系统根据 Jakarta EE 规范开发，以 Servlet 为控制核心，处理 HTTP 请求，解析参数，调用相应业务逻辑，采用 JSP 技术做为前端展示方式，利用服务器端数据实时绑定的技术来更新客户端的界面。

本系统使用业务功能模块化设计框架来创建，按照用户认证、内容推荐、音乐检索、行为跟踪等主要功能单元进行模块划分。在数据访问对象（DAO）层中，把各重要数据库表操作逻辑做统一抽象封装，较好地实现了业务逻辑和底层存储之间的解耦。采用 C3P0 连接池技术对资源进行管理，既可以实现连接池之间的共享，也可以对连接池的生命周期进行智能管理，大大降低了数据库连接所耗费成本。同时使用 Jedis 连接池，并加上容错功能，当出现 Redis 集群失败或者性能不足时，可自动切换到传统关系型数据库查询方式上，从而大幅提升系统高可用、稳定性的表现。

Jakarta EE Servlet 规范因长期在企业级开发领域被广泛使用而表现出很强的稳定性、可靠性能，对 Tomcat 容器有较强兼容性，MVC 架构可降低各模块之间耦合程度，为系统功能升级提供风险隔离，C3P0 连接池采用预加载数据库资源的方式，在高并发环境下可有效提高每条请求的处理速度和整个系统的资源利用效率。

2.7 本章小结

本章对个性化音乐推荐系统的全部核心技术进行详细解析，从建立完善的基础设施到构建各种重要的算法模型，形成了完整框架。在大数据处理阶段主要论述基于 PySpark 框架的分布式计算平台高效支持大规模特征提取和机器学习模型训练任务，使用 Redis 内存数据库完成热点内容快速缓存，显著加快在线服务响应速度。对于召回策略部分，介绍了 ALS 因子分解算法中加入置信度权重机制，并阐述了高维稀疏矩阵降维优化的关系和 FAISS 库中 IVFPQ 索引结构设计思想，在处理百万级候选项查询时依然可以维持毫秒级别的延迟水平。对排序建模部分，采用 LightGBM 决策树算法进行初筛筛选，使用 DeepFM 网络结构进行多模态特征融合模式的探索，利用 BST 时空序列分析模块捕捉用户行为随时间变化的趋势规律。最终采用两阶段 Stacking 集成框架将 DeepFM 与 BST 的预测结果融合，通过 5 折交叉验证生成堆外

预测，以逻辑回归元学习器学习两个基学习器的最优融合策略，在提升集成泛化能力的同时有效控制过拟合风险。前端交互界面采用 MVC 设计模式，将业务逻辑层、表现层和数据访问层进行分离，使用 C3P0 连接池来管理数据库的连接资源池，合理处理海量用户的并发请求。上述分布式计算、向量检索、排序建模与集成学习的理论方法，共同构成本系统后续设计与实现的算法基础，为第四章和第五章的工程落地提供理论依据。

3 系统需求分析

3.1 系统总体目标

AI 68%

本系统的设计目标是构建一套面向在线音乐场景的双模块个性化推荐平台，由 MusicWeb 用户交互模块与 MusicMode 推荐引擎模块协同构成。系统需在完整支持音乐播放、收藏、搜索等基础业务功能的同时，实现基于大数据技术的用户兴趣建模与个性化内容推送，并通过隐式反馈采集形成推荐优化的持续数据闭环。

AI 62%

从功能层面看，系统须为普通用户提供注册登录、音乐检索与播放、收藏与歌单管理、个性化推荐展示及账号自助管理等完整使用路径。对管理员则提供用户账号管理、歌曲资源管理、申诉审核与数据统计等后台运营能力。从而为推荐引擎提供多路候选召回、多模型精排融合与定时推荐更新等算法执行能力，三者之间通过共享的 MySQL 数据库实现数据流转与状态同步。

AI 70%

从性能与可用性层面看，系统在正常并发负载下须保证用户界面的交互响应延迟维持在可接受范围内。推荐引擎须在合理时间窗口内完成全量用户的每日推荐重算，向量检索须满足在线服务的响应时延要求。同时，系统须具备基本的异常容错与服务降级能力，推荐服务在部分模型异常时须能回退至基础召回策略，保证推荐列表的最低可用性。

AI 67%

综上，本系统总体目标可归纳为：以大数据技术为基础设施，以多模型集成精排管道为算法核心，以 Java Web 平台为用户交互载体，构建一套具备功能完整性、性能可达性与工程可维护性的个性化音乐推荐系统。

3.2 功能性需求分析

3.2.1 用户端功能需求

用户端以普通注册用户为主，建设一个全方位的音乐消费互动平台。根据的整体架构设计，可以将功能模块分为如下六个互相联系、共同工作的部分。各个功能模块之间用系统的业务流程来整合，以便更好地满足用户全流程的使用体验，其整体功能

如图 3-1 所示。

（1）账号管理

系统须支持新用户完成账号注册，注册信息包括用户名、密码、邮箱、手机号等基本资料。注册成功后，用户通过身份验证登录系统，系统维护用户的登录会话状态。用户在登录期间可访问个人中心，对昵称、邮箱、手机号等资料进行修改，并支持密码变更与账号注销操作。针对账号因违规被冻结的情形，系统须提供申诉入口，用户可填写申诉原因与联系邮箱提交申诉请求，管理员审核通过后须恢复账号状态并通知用户。

（2）音乐检索与浏览

AI 63%
系统须提供全文检索功能，支持用户按歌曲名称、艺术家、专辑、流派等多维度关键词进行组合搜索，并自动保存最近的搜索历史以便快速重复检索。在内容浏览层面，系统须提供热歌榜、新歌榜与收藏榜三类排行榜，按不同热度维度聚合展示全库曲目。

（3）音乐播放

AI 60%
系统须支持用户对目标曲目发起在线流媒体播放，播放器须具备进度拖拽、音量调节等基础交互控制能力。系统须支持多平台音源接入，通过统一的音频链接解析接口屏蔽底层平台差异，扩展可播放曲目范围。系统须实时采集用户的播放时长数据，作为隐式反馈信号上报至服务端以供推荐引擎使用。

（4）收藏与歌单管理

系统须支持用户对单曲执行收藏与取消收藏操作，并在页面上实时反映收藏状态。除单曲收藏外，系统须提供用户自定义歌单功能，允许用户创建多个歌单并向歌单中添加或移除歌曲，同时支持歌单名称与描述的编辑及歌单删除，并限制同一首歌重复加入同一歌单。每位用户在注册时须自动生成默认收藏歌单，作为收藏行为的统一承载容器。

（5）个性化推荐展示

AI 64%
系统须在用户登录后的主页面展示由推荐引擎生成的个性化推荐列表，推荐结果

须实时从数据库中读取，并支持用户手动触发推荐刷新。推荐列表须集成播放与收藏操作入口，以便在展示场景下直接完成用户行为的闭环采集。

（6）播放历史查询

系统须记录用户的全量播放行为，并以时间倒序提供分页浏览能力，用户可通过播放历史页面回顾历史播放曲目。

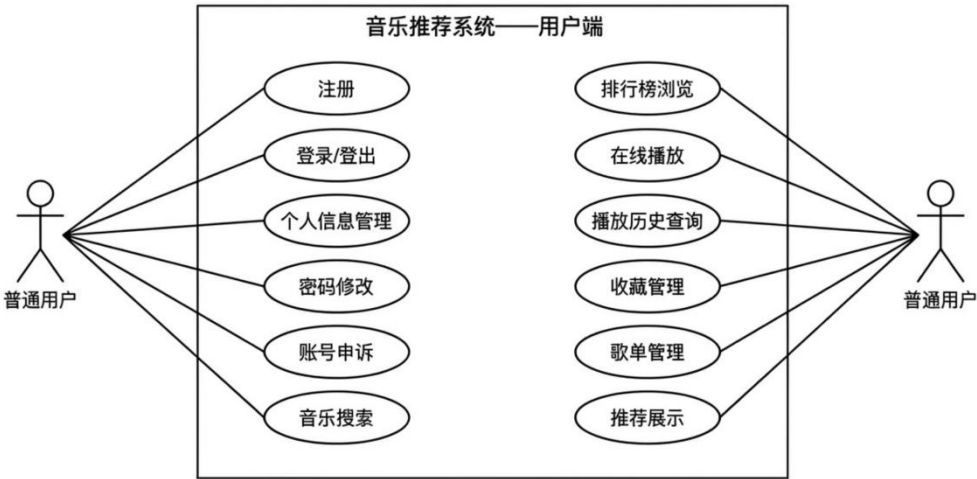


图3-1 用户端用例图

3.2.2 管理端功能需求

管理端专门给系统管理员服务，主要搭建起平台运维和内容监管的后台支持架构。经过唯一的身份认证及多次的安全审核之后，才可以让管理员进入到专用的管理界面当中去。本模块从四个方面进行规划，然后用具体单元的形式来完成部署工作。整体功能如图 3-2 所示。

（1）数据概览仪表盘

管理后台须提供实时数据统计视图，汇总展示当前注册用户总数、系统入库歌曲总数、全局收藏记录数量等平台核心运营指标，辅助管理员掌握系统整体运行状态。

（2）用户账号管理

AI 66%
管理员须能够查看全量注册用户列表，支持按用户名、注册时间等条件进行搜索与排序。针对存在违规行为的用户，管理员须能够执行账号冻结操作，指定冻结截止时间与冻结原因，并且对于状态正常的账号，管理员须能解除冻结限制。系统须通过

权限控制机制确保普通用户无法访问管理端的任何接口与页面。

（3）申诉审核管理

管理员须能查看所有待处理的账号申诉记录，对每条申诉作出同意或拒绝的处置决定，并可填写回复说明。系统须在申诉处置完成后通过邮件服务自动向用户发送审核结果通知，当申诉被批准时，系统须同步恢复该账号的正常访问状态。

（4）歌曲与收藏管理

AI 66%
管理员须能够浏览系统全量歌曲库，查看每首歌曲的元数据信息，并支持新增、编辑与删除歌曲记录。针对用户歌单，管理员须能查看所有用户创建的歌单及其包含的歌曲，并支持对违规或异常歌单执行删除操作以及从歌单中移除特定歌曲。

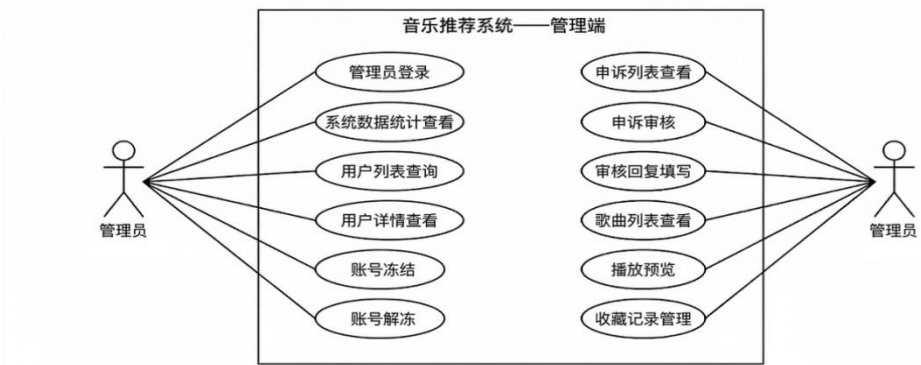


图3-2 管理员端用例图

3.2.3 推荐引擎功能需求

推荐引擎模块（MusicMode）作为系统的算法核心，承担从候选生成到最终推荐写库的全流程计算任务，其功能需求覆盖以下五个方面。

（1）多路并行候选召回

本研究设计出三个独立运行的候选推荐模块，第一个模块依靠 FAISS 库搭建起向量空间近似检索框架，借助用户画像特征同大批量音乐嵌入数据的相似度对比，从大量的索引当中挑选出高潜力的目标。第二个模块采用 ALS 协同过滤算法，将用户行为矩阵进行隐含因子分解从而挖掘出个体喜好，进而产生个性化的备选名单。第三

个模块当作兜底方案，在极端的冷启动情况或者新用户的激活过程中使用热度驱动机制，结合内容的流行度和创新性评分来给用户供应多种选择。经过上述模块融合得到多源候选集，再进行去重优化以及优先级排序之后得到最终的推荐流程闭环。

（2）多模型异构精排

AI 58%

候选集须经粗排与精排两个阶段逐级筛选。在粗排阶段，系统须调用 LightGBM 对全量合并候选进行基于手工特征的快速评分，筛选出置信度较高的子集。在精排阶段，系统须以 DeepFM 与 BST 双模型对粗排结果分别进行高阶特征交叉建模与用户行为序列建模，两模型的输出评分须通过离线 Stacking 优化得出的权重系数进行加权融合，得到最终集成评分并筛选出推荐候选集。

（3）多样性重排

利用最大边际相关性（MMR）算法做多元化排序的时候，要考虑到候选音乐作品同用户偏好之间的关联强度，还要考虑到它同其它推荐项目之间存在的差别，进而有效克服个性化推荐列表里的内容重复状况。对于某一类别的作品数量进行限定，在保证结果集中艺术表现形式丰富的同时也能避免出现单一化的趋势。

（4）隐式反馈采集与冷却机制

该系统应该将每一次推荐的音乐作品的信息存入到反馈记录表里，在用户播放、收藏等互动操作之后，及时对相关条目更新播放次数和时长，更新收藏标志。对于因为多次负面交互而被评定为低质量的歌曲，应该启动降权隔离措施，对它进行指定周期内的屏蔽，从而防止内容污染的出现。

（5）定时推荐更新调度

系统须支持通过定时调度机制在每日指定时间自动触发全量推荐重算，确保每日向所有注册用户更新个性化推荐列表，并将推荐结果批量写入数据库供前端实时读取。

3.3 非功能性需求分析

3.3.1 性能需求

（1）交互响应延迟

AI 63%

客户端界面在得到服务端的数据之后必须对延迟进行严格的控制，保证用户交互过程的顺畅。对高频只读查询场景比如音乐搜索、排行榜信息检索等，可以利用 Redis 缓存技术提前将结果进行预加载处理。该策略一方面可以明显减小数据库的负担，另一方面可以提高核心业务逻辑服务速度，从而加强系统性能及稳定程度。

（2）推荐计算吞吐能力

推荐引擎在执行每日全量推荐重算时，须能在预设的定时窗口内完成面向全量注册用户的三通道候选召回、多模型排序评分与推荐结果写库操作，保证每日推荐更新的计算吞吐能力。

（3）数据库并发连接能力

系统须通过 C3P0 数据库连接池统一管理并发数据库连接，在高并发请求峰值期间弹性扩展可用连接数，保证请求不因连接资源耗尽而长时间阻塞，而访问低谷期则自动回收空闲连接，降低数据库端的持续资源占用。

3.3.2 安全性需求

（1）注入攻击防护

在数据库操作中要采用参数化查询的方式把用户的输入数据和 SQL 语句的逻辑分开来，从而降低 SQL 注入的风险。前端表单要集成格式校验、长度约束等一系列复杂的防护功能，保证在用户提交数据的时候不会出现问题。

（2）跨站脚本防护

系统在将用户提交的内容回显至页面时，须对特殊字符进行转义处理，防止 XSS（跨站脚本攻击）将恶意脚本注入其他用户的浏览器上下文，保护用户会话数据的安全性。

（3）身份认证与权限隔离

该系统应该使用服务端会话管理的方式来保存用户的登录状态，对于包含权限控制的页面或者接口，在服务器端要对会话进行严格的验证，不能让非法访问进入。为了达到基于角色的细粒度权限管理的目的，在管理后台和普通用户端都要设置访问控

制的安全屏障，管理员账户需要独立身份认证区分，而且只能使用到管理员功能模块中的功能。同时也要对服务器端的权限进行严格地权限映射规定，从而保证服务器端的安全以及业务的合法性。

（4）账号状态管控

系统须对被冻结或注销账号的用户拦截其登录请求，并向其展示明确的账号状态说明与申诉入口，防止异常账号继续访问受保护资源，以及管理员对账号的冻结与解冻操作须即时生效。

3.4 本章小结

AI 70%
本章围绕系统建设目标与具体需求规格展开详细分析，为后续的架构设计与工程开发提供了明确指引。在总体目标层面，确立了基于前端交互 MusicWeb 与推荐引擎 MusicMode 双模块解耦协同的总体系统定位。在系统的功能需求层面，分别从普通用户、系统管理员与推荐引擎三个执行主体的视角进行了业务用例梳理：用户端涵盖了账号基本信息管理、多源音乐检索检索与播放、个性化音乐推荐展示及自定义收藏歌单等核心交互功能；管理端界定了全局统计数据概览、用户信息统一管理、账号申诉状态审核及歌曲资源库管控等运营能力要求；推荐引擎端则明确了多路候选并行召回、两阶段异构精排融合、多样性算法重排过滤及隐式反馈信号采集等算法计算任务。在非功能需求层面，对前端请求交互响应时延与推荐引擎吞吐能力等性能表现指标，以及平台 SQL 参数注入防护、会话权限状态隔离等系统安全性要求进行了明确规定。上述通过多维度对相关软硬件需求的综合推演与严密剖析，从业务逻辑与性能指标两个方向清晰界定了该个性化音乐推荐系统的实施边界，为第四章系统整体架构规划、推荐管道梳理以及业务数据库结构建立奠定了坚实的理论依据与需求牵引。

4 系统设计

4.1 引言

AI 67%

在第 3 章明确了系统的功能抽象边界与非功能需求的基础上，本章旨在将宏观的在线推荐与各项交互业务需求转化为具体的技术蓝图框架，完成系统建设初期由明确需求向架构设计的平滑转变。本章将重点围绕系统的总体双端协作架构、核心推荐算法推演流水线、底层中心数据库关联结构以及面向用户的前端交互逻辑四个主要领域进行深入探讨与架构搭建。最终经由本章梳理产出的各项技术机制架构与数据质量约束方案，将为第五章各项功能业务的代码工程落地过程提供明确的系统实现路径与指导思想。

4.2 系统总体架构设计

AI 58%

本系统采用双模块解耦架构，由面向用户交互的 MusicWeb 模块与面向算法计算的 MusicMode 模块协同构成，两者以 MySQL 关系型数据库为共享数据层进行集成，通过异步批处理机制实现数据闭环。如图 4-1 所示，系统整体架构由左侧 MusicWeb、右侧 MusicMode 及底部共享数据层三部分组成：MusicWeb 持续将用户行为信号写入共享数据库，MusicMode 周期性从数据库读取行为数据完成推荐计算后将结果回写，MusicWeb 再从数据库读取推荐结果渲染至用户界面，形成完整的数据驱动推荐闭环。

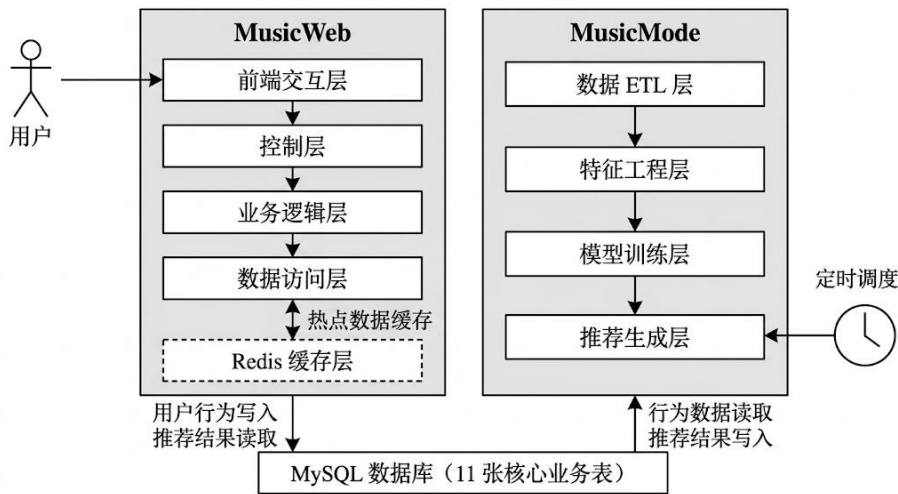


图4-1 MusicWeb + MusicMode双模块整体架构图

4.2.1 MusicWeb 前端模块架构

MusicWeb 是系统的用户交互平台，使用 Java 和 Jakarta Servlet/JSP 技术栈来建立，其架构为 MVC 五层架构，由视图层、控制层、业务逻辑层、数据访问层、实体层五部分组成，各个部分之间职责分明，层次之间互相依赖关系只有一层，即下层向上传递。

（1）视图层

视图层由 JSP 动态页面、JavaScript 行为脚本、CSS 样式表三部分组成，用来向用户浏览器输出可以交互的 HTML 界面。页面集合包括首页、注册页、用户中心、歌曲搜索、歌单详情、播放历史、用户设置和账号申诉等业务页面，还有管理员后台的仪表盘、用户信息、歌单管理、歌曲管理、申诉审核等运营页面。前端层利用异步请求与控制层进行交互，可以实现推荐列表刷新、收藏状态变更等无刷新的动态操作，从而减小由于整页加载而给用户造成的感知延迟。

（2）控制层

控制层是由一组根据业务功能独立出来的 Servlet 控制器所组成，它集中地将表现层发出的 HTTP 请求传送到下一层（JSP 层）上去执行。根据业务域来划分控制器集合，共包含用户认证、个人信息管理、音乐服务、行为上报、内容管理、推荐服务、管理端这 7 个功能域。控制层不会直接去处理数据库的操作，只会对参数进行解析，对权限进行校验，并且会安排流程的调度工作，从而保证自身的轻量化。

（3）业务逻辑层

业务逻辑层负责承载跨 DAO 的复合操作与推荐服务的协调逻辑。推荐服务组件负责从数据库读取由推荐引擎预计算的推荐结果，并结合用户当前的收藏状态与历史播放记录对推荐列表进行过滤与补全，最终向控制层返回可直接渲染的推荐歌曲列表。

（4）数据访问层

数据访问层用 C3P0 连接池来获得数据库连接，对系统全部 11 张核心业务表的读写操作做统一封装，将 SQL 语句的细节和结果集的映射逻辑都隐藏起来。各个数据访问组件按照业务实体分成了用户信息、歌曲元数据、播放历史、收藏关系、歌单

数据、推荐记录和反馈数据这些主要实体的持久化工作，全部 SQL 操作都采取了参数化查询的办法来规避 SQL 注入这种安全问题。此外，数据访问层还通过 RedisUtil 组件对外提供缓存读写接口，将用户收藏状态等读多写少的热点数据存储于内存，读请求优先命中缓存，写请求在更新数据库的同时使对应缓存键失效，保证缓存与数据库的一致性。

（5）实体层

AI 58%

实体层由各核心业务实体类构成，涵盖用户、歌曲、歌单、收藏关系、播放历史与申诉等业务对象，与数据库各核心业务表一一对应。各实体类仅封装字段属性与访问器方法，不含业务逻辑，在控制层、业务逻辑层与数据访问层之间作为统一的数据载体流转，避免各层直接依赖松散的键值结构，从而降低层间耦合程度。

4.2.2 MusicMode 推荐引擎模块架构

MusicMode 为系统的算法计算部分，使用 Python 和 PyTorch 的深度学习库建立，并采用定时批处理的方式运行。模块内部自上而下分为数据 ETL 层、特征工程层、模型训练层和推荐生成层四个功能层次，各个层次的输出物通过磁盘序列化文件或者数据库表来实现跨层传递，并且可以支持各个层次的独立执行以及增量更新。

（1）数据 ETL 层

ETL 层是将 KKBOX 公开数据集和 MySQL 业务数据库里的原始数据抽取出来，对它们进行数据清洗、格式规范化以及样本的构造。对于 KKBOX 大规模的歌曲数据集，使用 PySpark 分布式计算框架用 JDBC 接口对全量数据进行批量导入，然后根据歌曲唯一标识做增量去重，防止重复写入。对于模型训练样本的构建，ETL 层把用户 30 天内重复收听的行为当作正样本的标准，用等比例随机负采样的方式得到负样本，最后输出结构化的训练样本集给特征工程层使用。

（2）特征工程层

特征工程层对 ETL 层输出的原始样本进行特征提取工作，给三个模型分别创建出互不相关的特征集合。LightGBM 的特征集有 59 个，包含用户行为统计、歌曲属性

统计、用户-歌曲交互匹配、时序特征、近期交互信号、SVD 嵌入向量等特征组；DeepFM 的特征集为 50 个；BST 的特征集为 58 个，在 DeepFM 稀疏特征的基础上又增加了序列行为特征，可以进行用户历史交互序列时序建模。所有类别型特征的编码都用 OOF 折外预测的方法，在 K 折交叉验证的框架内，每折的编码值是由其余 K-1 折的样本统计得到的，在全量训练集的基础上，检验集和测试集的编码值通过统计计算得出。

（3）模型训练层

模型训练层采用的是 LightGBM 做为粗排模型，用 59 维的特征集做独立训练。DeepFM、BST 为精排层的两个基模型，分别使用不同的特征集进行 GPU 加速来完成深度模型的训练。精排集成阶段用 DeepFM 和 BST 在验证集上的折外预测概率作为元特征，使用 SLSQP 约束优化算法求解最优融合权重，权重非负并且总和为 1，保证了集成结果的概率可解释性。各模型的权重文件以及集成配置都被序列化到磁盘上，可以供推荐生成层使用。

（4）推荐生成层

推荐生成层完成四层推荐管道的完整在线执行逻辑，按照三通道候选召回、LightGBM 粗排筛选、DeepFM 和 BST 双模型加权集成精排、多样性重排的顺序依次处理，最后得到每个用户的个性化推荐列表并批量写入数据库。

4.2.3 双模块交互与部署设计

AI 51%
MusicWeb 与 MusicMode 两个模块之间采用共享的 MySQL 数据库作为唯一数据交换媒介，建立了一种典型的异步松耦合集成架构。该设计具备三项工程优势：其一，得益于接口调用直接依赖的剥离，两模块支持单独的部署与重启，实现了运行故障的互相隔离；其二，数据库充当了持久化数据转储载体，即使推荐引擎在批处理计算中途异常中断，前端依然能连贯读取已存库的昨日推荐列表；其三，运维人员直接检索数据表即可观测跨模块信息的实际流转情况，降低了排障分析难度。

AI 69%
在系统部署层面，MusicWeb 被构建为标准 war 包运行于 Apache Tomcat 容器中，

作为常驻应用提供实时交互访问。**MusicMode 算法引擎**则以 **Python 脚本群**形态存在，由操作系统的定时任务按日周期触发全局更新重算。底层的 **Redis 缓存**与 **MySQL 库**分别以独立服务进程驻留，项目整体依托专用批处理脚本负责各项接口的就绪探测及顺序拉起工作，以此确保前端层对外接收请求之前，应用所需的全部前置组件均已挂载完毕。

4.3 推荐算法流水线设计

推荐算法流水线是本系统实现千人千面个性化推荐的主要方式。图 4-2 为流水线的结构，为四层串行结构，从上到下分别是召回层、粗排层、集成层和重排层。各层有各自的颗粒度候选筛选任务，召回层用高覆盖的目标，对全量曲库并行产生大约 600 个候选；粗排层以效率为第一要务的梯度提升模型，快速给候选打分排序压缩；集成层用 DeepFM 和 BST 两个模型异构融合，对精选出来的候选做高精度的精排；重排层在最终输出之前加入多样性约束，避免推荐列表内容雷同。四层管道分工清楚、相互关联，保证各个层次可以独立进行迭代而不会对其他层次造成干扰。

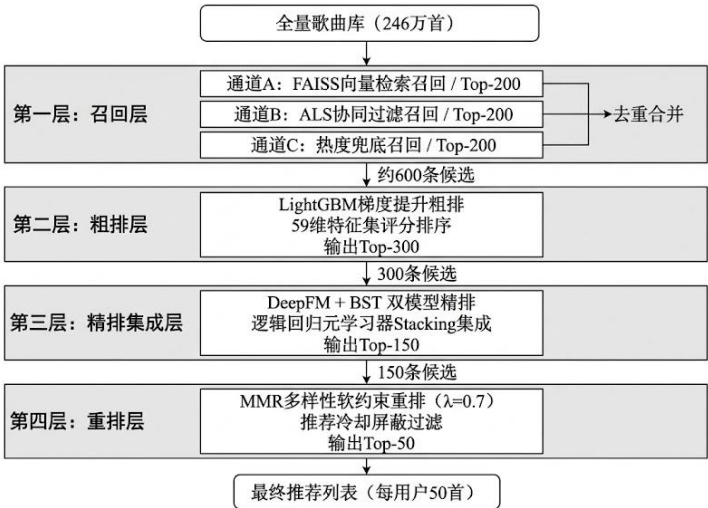


图4-2 四层推荐管道总体流程图

4.3.1 召回层设计

AI 70%

召回层的核心设计目标是在计算效率约束下实现对高质量候选歌曲的高覆盖率

捕获。系统设计了三条相互独立、特性互补的召回通道，通过并行执行后去重合并的方式构建统一候选集。如图 4-3 所示，三通道分别基于向量近邻检索、协同过滤与热度排序三种不同的信息源进行候选生成，以覆盖不同类型的用户兴趣信号。

通道 A：FAISS 向量检索召回

根据用户行为的历史建立用户兴趣向量，用 FAISS 的内积索引做全量歌曲向量空间的精确内积搜索，召回和用户兴趣向量内积相似度最高的 Top-200 个候选歌曲。该通道设计逻辑是用户历史播放偏好的语义嵌入空间可以被有效地压缩成一个固定的维度——兴趣表示向量，与这个向量距离近的歌曲在语义特征分布上和用户偏好高度一致，因此被播放的概率较高。向量索引在离线阶段就完成了全量歌曲库的预计算和持久化，而在线召回阶段只需要做向量查询的操作，单次查询效率较高，可以满足大规模推荐场景下吞吐量的要求。

通道 B：ALS 协同过滤召回

AI 68%

协同过滤通道基于 ALS 隐式反馈矩阵分解，捕捉用户间的协同信号，向每位用户提供 200 首候选歌曲。为应对用户行为数据稀疏性差异，系统按活跃程度设计三级降级路由：高活跃用户（ALS 训练集有效播放次数不低于 10 次）直接使用离线训练所得隐因子向量做内积排序召回；中等活跃用户（收藏数不低于 5 首或播放次数不低于 10 次但不满足前者）改用多维内容相似度加权召回；冷启动用户则退回注册时填写的流派与艺术家偏好做兜底召回。ALS 矩阵分解在离线阶段完成，隐向量序列化存储后供在线直接读取，满足高吞吐、低延迟的召回要求。

通道 C：热度兜底召回

热度兜底通道以歌曲总播放量和用户偏好艺术家加权分的组合形式来排序，从中选出前 200 首歌曲进行补充。该通道用来解决冷启动场景下推荐的可用性问题，对于行为信号极度稀疏的新用户，当向量检索和协同过滤通道失效的时候，热度兜底依靠平台级的高热内容来保证推荐列表的基本质量和时效性。对于活跃用户，将上面提到的通道结果去除重合部分合并后得到候选集，从而扩大了候选集的覆盖范围。系统构建出完备的多层召回机制，在数据环境十分恶劣的时候，使推荐引擎具有很强的鲁棒

性。

三个通道分别生成候选列表之后，系统用歌曲唯一标识做键进行集合去重操作，剔除掉用户最近已经播放过的以及处于推荐冷却期的歌曲，最后合并得到约 600 个不重复的候选集，传送到下游精排层。

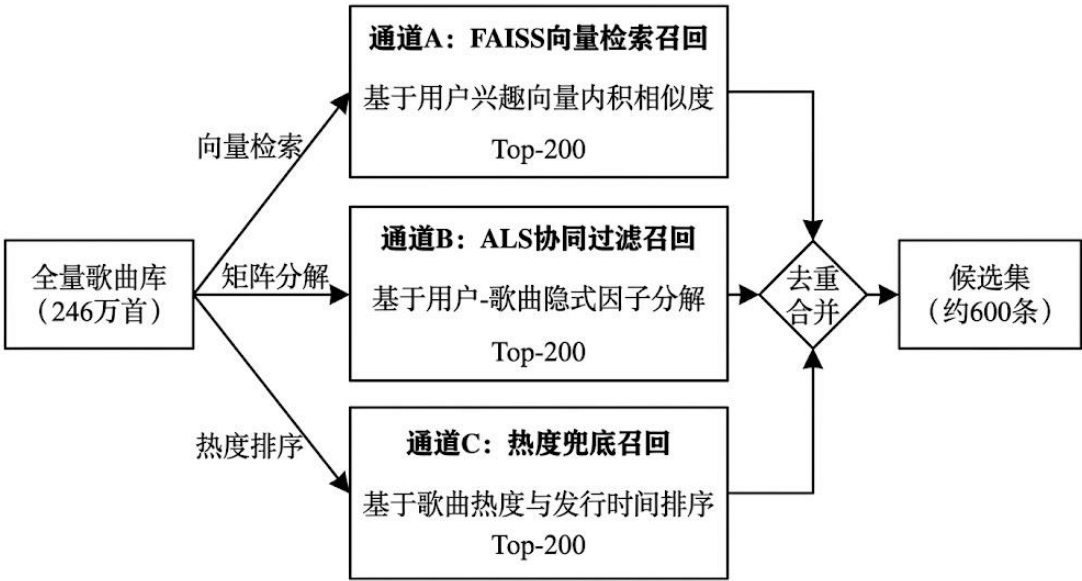


图4-3 三通道并行召回结构图

4.3.2 粗排层与精排层设计

召回层用高覆盖率的目标输出大约 600 首候选，但是候选间的相对优劣还没有得到细致的区分。因此粗排层将以效率为先，用轻量级模型对全部候选进行快速过滤，精排层以精度为先，用两个深度模型分别对过滤后的候选进行评分。两层分别把候选规模从大约 600 首减小到 300 首，并且把精排层给集成层供应两路评分输入，从而达到从众多数据向准确预估的逻辑转变，保证系统既具有较快的推断速度又具备较高的推荐质量。

其中粗排层用 LightGBM 梯度提升模型对所有的约 600 首候选执行快速评分，按照预测播放概率降序选出 Top300 传送到精排层。本文选择 LightGBM 的原因是其采用直方图近似算法来进行决策树分裂，而不是像深度神经网络那样使用矩阵乘法和激活函数运算，从而在大量候选推断的情况下可以将延迟控制在很低的程度。另外，

DeepFM 和 BST 深度模型计算开销较大，如果对全部候选进行推断，会大大加重系统的负担，因此只对粗排后的 Top300 子集进行推断。粗排层使用独立的 59 维特征集，在保证推断效能的基础上给精排层提供了一个很好的输入来源。

而精排层就是对粗排结果前 300 个候选曲分别使用 DeepFM、BST 两个深度模型独立打分。DeepFM 利用共享嵌入层来同时建模低阶因子分解特征的交叉以及高阶非线性变换，可以捕捉到手工特征没有显式包含的隐性属性组合，从静态偏好角度给出评分。BST 用多头自注意力机制对用户的过去播放序列做时序 Transformer 编码，保留行为信号的时序信息，可以动态地察觉到兴趣的发展变化，在序列偏好角度给出评分。两个模型从静态属性匹配和动态序列偏好两个互补的角度出发，对于同一批候选分别给出独立的概率分布，最终传入集成层完成加权融合，从而达到深度互补的目标需求和高精度的推荐结果。

4.3.3 集成层设计

AI 68%

集成层承接精排层输出的 DeepFM 与 BST 两组独立评分，通过两阶段 Stacking 元学习器对其进行融合，按集成得分降序截取前 150 首传递至重排层，其中集成权重通过离线 Stacking 框架学习。

在离线训练阶段，DeepFM 与 BST 分别基于全局播放时间戳进行 5 折时序交叉验证，生成各自在验证集上的折外预测概率。用两组折外预测向量组成元特征矩阵，以验证集真实标签为目标，训练逻辑回归元学习器，拟合最终融合概率，训练所得元学习器权重参数序列化存储，供在线阶段直接加载。

AI 58%

在线推荐阶段，集成层对前 300 所候选中的歌曲进行读取，然后精排层输出两组评分概率，按式(2-11)计算集成推荐得分，接着按集成得分降序截取前 150 首传递至重排层。至此，集成层输出的 150 首候选在预测播放概率层面已经过充分筛选，以供重排层进行进一步的多样性约束筛选。

4.3.4 重排层设计

重排层在集成层得到的 150 首候选之上，利用 MMR 算法做多样性重排、剔除当

前处于推荐冷却期的歌曲，再把候选量减到每个用户 50 首。重排层属于四层管道里的最后一步，其主要任务是在保证集成评分准确性的前提下，依照分布均衡的要求来改善推荐列表的内容，防止出现因为模型有偏从而产生的同质化呈现的情况发生。

(1) MMR 最大边际相关重排

集成评分以单曲维度的播放概率为优化目标，仅对候选歌曲逐一独立评分，不对推荐列表的整体内容分布加以约束。重排层采用 MMR（最大边际相关，Maximal Marginal Relevance）算法，在候选歌曲的相关性与已选列表的多样性之间进行权衡，其得分公式如式(4-1)所示：

$$MMR(d) = \lambda \cdot Rel(d) - (1 - \lambda) \cdot \max_{d_j \in S} Sim(d, d_j) \tag{4-1}$$

- 式中 d——当前待评估的候选歌曲；
- Rel(d)——该候选歌曲的归一化集成评分，取值范围为[0,1]；
- Sim(d,d_j)——候选歌曲 d 与已选列表中歌曲 d_j 基于 FAISS 嵌入向量计算的余弦相似度；
- S——已选列表，即当前已通过贪心选取加入推荐结果的歌曲集合；
- λ——相关性权重系数，取 λ=0.7，即相关性占 70%，多样性惩罚占 30%。

算法按照 MMR 得分贪心地选取每一轮候选中 MMR 值最高的歌曲加入已选列表，直到选满 50 首为止。此外，系统对同一艺术家的入选数量设有上限作为兜底保护，超出上限的候选不做直接剔除，而是对其 MMR 得分施加惩罚系数后重新参与排序，在保障多样性的同时避免因强制过滤而损失高相关性候选。

(2) 推荐冷却过滤

系统对每一个用户与歌曲的互动都会有一个冷却结束时间，对于连续出现一次或者多次负向评价的歌曲，会设置一定天数的冷启动推荐限制。重排层在贪心筛选时实时过滤掉冷却期内的候选歌曲，防止用户看到过的内容再被曝光。重排层只做只读过滤操作，冷却策略触发条件和时长由上游反馈处理逻辑来计算，各层职责只有一条。经过多样性重排和冷却过滤之后，最终得到每位用户 50 首推荐歌曲，以确保既

具备评分的准确性又保持内容的多样性，然后将其存储到数据库的推荐表中，以供用户访问时可以即时读取并展示给用户，从而形成一个完整的推荐闭环。

4.4 数据库设计

4.4.1 表的概念结构设计

概念结构设计阶段用实体-关系模型（E-R 模型）把系统业务数据加以抽象，清楚地体现出各个实体的属性及其之间的联系，为后面逻辑结构的设计打下基础。

本文共准备设计 11 个主要实体，分别是用户表、歌曲表、播放历史表、用户歌单表、歌单歌曲关联表、推荐结果表、推荐反馈表、用户满意度反馈表、申诉表、用户屏蔽内容表和歌曲滚动统计表。

各实体间的核心关系如下：用户与歌曲之间是多对多的关系，一个用户可播放多首歌曲，一首歌曲可被多个用户播放，每条播放行为均以独立记录的形式存入播放历史表；用户与歌单之间是一对多的关系，一个用户可创建多个歌单，歌单与歌曲之间通过歌单歌曲关联表建立多对多的关系；用户与推荐结果表之间是一对多的关系，系统每天为每个用户生成若干推荐记录，推荐反馈表附着在推荐结果表上，记录用户对推荐歌曲的真实行为；用户满意度反馈表、申诉表、用户屏蔽内容表均以用户为中心，分别记录偏好评价、违规申诉与内容屏蔽信息。歌曲滚动统计表与歌曲主表是一对一的关系，每首歌曲对应一条预聚合的滚动统计记录，以独立表的形式存储近 7 天、近 30 天的播放量及热度趋势指标，防止频繁统计更新影响歌曲主表的查询性能。系统主要实体间的关系如图 4-4 所示。

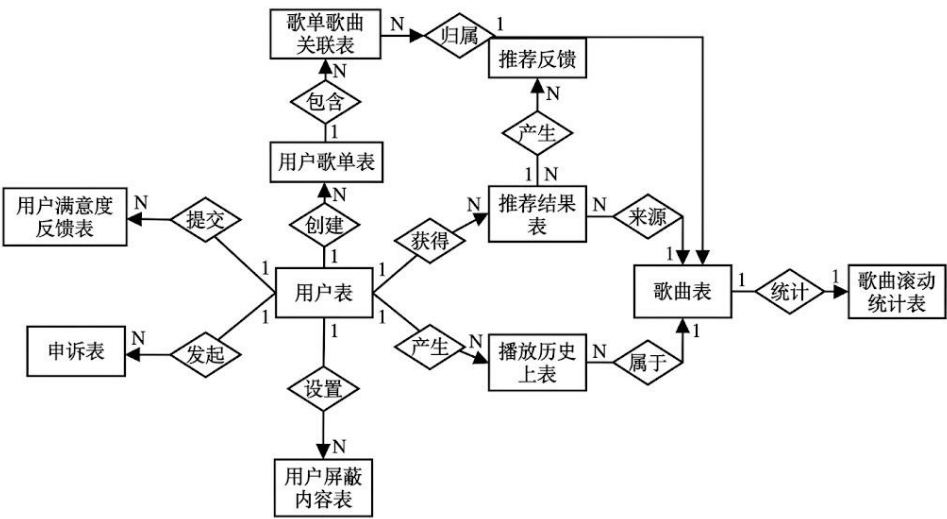


图 4-4 系统核心实体关系图

4.4.2 表的逻辑结构设计

在对每个表的结构设计阶段，将概念模型中的 11 个实体分别对应到 MySQL 数据库中 11 张业务表上，使用外键约束和联合索引来保证数据的一致性以及查询的速度。各个表的详细字段结构详见附件 1。

11 张表具体的作用分别为：用户表存储全部注册用户的账号信息和状态标记；歌曲主表保存有 246 万首歌曲的全部元数据；播放历史表用时间戳保存每次用户-歌曲的交互事件，是特征工程和模型训练的基础数据；用户歌单表保存用户创建的歌单，歌单歌曲关联表实现歌单和歌曲的多对多映射；推荐结果表批量存储每天给每个用户推荐的 50 首推荐候选，推荐反馈表在上一步的基础上记录用户实际的交互行为和冷却状态；用户满意度反馈表记录用户对每天推荐的总体显式评价；申诉表接收用户提出的申诉申请；用户屏蔽内容表保存流派和艺术家的软屏蔽规则；歌曲滚动统计表预聚合每首歌曲最近七天和三十天的播放量和热度趋势，用独立表结构避免频繁写入影响歌曲主表的查询效率。

4.5 前端交互设计

4.5.1 页面总体结构设计

系统页面整体采用统一的头部导航布局。顶部导航栏在未登录状态下仅展示首页和登录引导文字。在已登录状态下扩展为首页、发现、排行榜、我的收藏四个链接，并在右侧显示用户头像、用户名、退出与设置入口。各页面样式由独立的 CSS 文件管理，主内容区依照当前功能模块动态渲染对应视图，实现样式隔离与可维护性。

系统页面分为两类：首页为公开访问的网站，展示宣传内容与新歌推荐，而已登录用户访问时将展示用户主页、搜索框、歌单详情、个人设置等功能页面，各页面在服务端通过 Session 检查用户登录状态，未登录用户访问时将被自动跳转至首页登录入口。系统整体页面导航层次如图 4-5 所示。

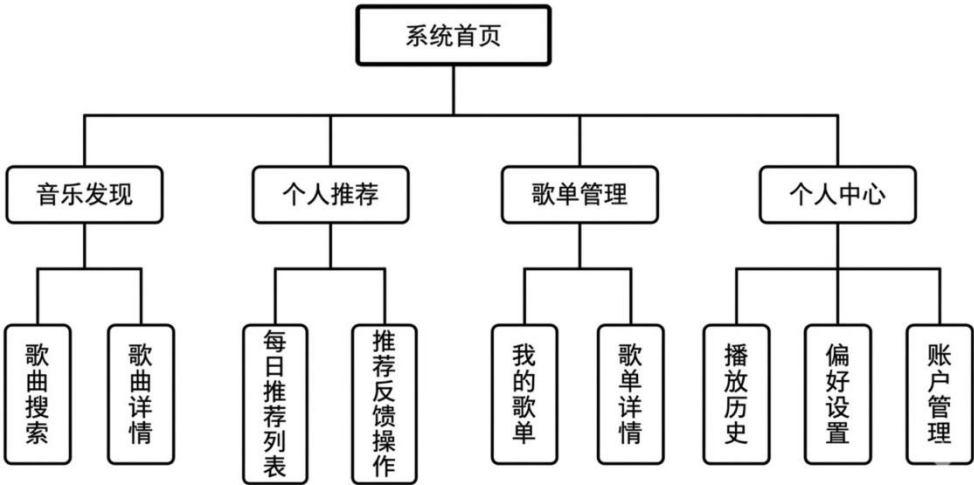


图4-5 系统页面导航层次结构图

图 4-5 以树形层次清晰呈现了系统各功能页面的组织关系。一级节点为系统的登录首页，二级节点为四大功能模块，三级节点为各模块下的具体页面。用户可通过顶部导航栏直达各二级模块，再经页面内部交互跳转至对应三级页面，路径清晰，交互层级不超过三级，有效降低用户的操作认知负担。

4.5.2 核心功能页面设计

系统核心功能页面共分为四个模块，各模块在交互设计上均以简洁高效为原则，力求以最少的操作步骤完成目标任务。

(1) 音乐搜索模块

音乐搜索页面由搜索输入区和搜索结果区两部分构成，搜索输入区采用居中的单行搜索框，输入框获得焦点时自动展开搜索历史下拉面板，面板支持一键清空；搜索框下方设有三个来源切换按钮，当前激活来源以高亮样式标注。搜索结果区以左侧色块的标题行展示命中数量，结果以列表行形式逐条排列，每行包含封面缩略图、歌名、来源平台标签、VIP 标签、歌手、专辑名和时长，行右侧提供加入歌单与播放两个操作按钮。点击加入歌单按钮后展开下拉浮层供用户选择目标歌单，无结果时展示居中的空状态图标与引导文字。

（2）个性化推荐模块

个性化推荐模块是系统展示核心界面，为用户展示根据推荐算法得到的每日推荐歌曲。页面用有序列表的形式显示当天 50 首推荐的歌曲，每一行显示歌曲的基本信息以及操作区（播放、收藏、反馈）。用户在本页面所有交互操作都会立即触发后台异步请求，把数据提交给后台之后再写入到推荐反馈表中，从而构成下一轮推荐迭代的训练输入。

（3）歌单管理模块

歌单管理模块支持用户创建、编辑和删除自定义歌单，以及向歌单中逐条添加或移除歌曲。用户主页以卡片网格的形式展示全部歌单，点击歌单卡片跳转至歌单详情页。详情页以分页列表的方式呈现歌单内所有歌曲，支持按添加时间、歌手、专辑、年份和播放次数进行排序，每条记录末尾提供移除入口。编辑与删除操作仅对自定义歌单开放，系统默认歌单不可编辑或删除。歌单数据通过数据访问层持久化至数据库，操作结果即时反映在页面上。

（4）个人中心模块

个人中心模块，以标签页形式预实现三项功能：个人信息标签页支持修改昵称、邮箱、手机号、性别与城市；账户安全标签页提供密码修改功能；账户管理标签页展示账号基本信息并提供账号注销入口。播放历史以独立页面呈现，用户可在用户主页跳转查看。内容屏蔽功能位于用户主页侧边栏面板，支持按流派和歌手设置屏蔽规则。

4.5.3 推荐交互与行为反馈设计

推荐交互设计的主要目的就是快速获取行为信号，并且形成一个不断改善的反馈闭环，如图 4-6 所示。推荐引擎离线完成候选生成和排序后将结果保存到数据库中，前端在用户访问的时候读取并展示当日推荐列表。用户在推荐页面触发的交互分为三类分支处理，播放行为触发前端计时器实时记录播放进度，用户离开时通过异步请求将完播率上报到后端，收藏行为作为强正向信号立即写入反馈记录，反馈不喜欢行为被标记为负向交互，触发相应内容的降权处理。三种行为一起放在推荐反馈表里，并且作为下一回合推荐改良进程的训练讯号。评分细节部分，完播率低于 20%的播放行为被记作跳曲，系统以此来对歌曲或者歌手之后的推荐做出降权处理；收藏行为是强正向信号，会增大相似风格内容的推荐权重。

另外用户可以在个人中心主动声明对某个流派或者歌手的屏蔽，系统使用递增冷却机制，即每一次屏蔽操作加上屏蔽计数，冷却到期日随着计数增加而延长，到期之后屏蔽状态自动失效，防止单次操作造成永久性的内容排除，以上各种行为反馈一起形成了系统推荐的在线学习闭环。

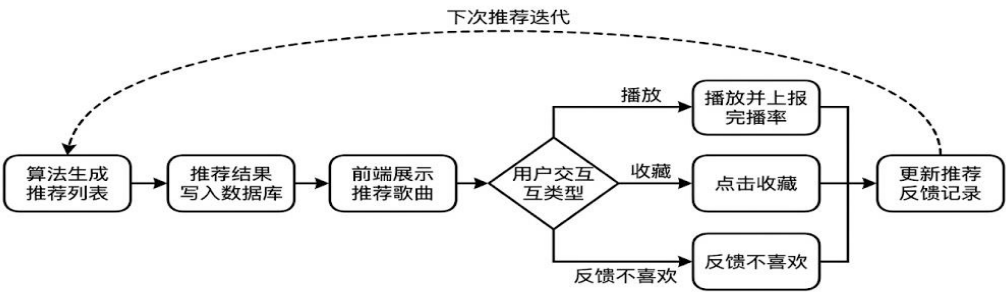


图4-6 推荐交互行为反馈闭环流程图

4.6 本章小结

本章在第三章需求分析所确定的功能边界与算法目标的基础上，完成了系统从"做什么"到"怎么做"的设计转化，围绕系统总体架构、推荐算法流水线、数据库结构、前端交互四个主要设计领域，形成了一份指导工程实现的技术蓝图。

AI 69%

在系统总体架构设计上，本章提出了 MusicWeb 与 MusicMode 双模块解耦架构。MusicWeb 基于 Java 与 Jakarta Servlet/JSP 构建，以前端层、控制层、业务逻辑层、数据访问层与缓存层五层 MVC 架构承载用户交互逻辑。MusicMode 基于 Python 与

PyTorch 构建，以 ETL、特征工程、模型训练与推荐生成四个功能层驱动算法计算。两模块以 MySQL 共享数据库为唯一数据交换通道，采用松耦合集成方式，支持独立部署与扩展。

AI 69%

在推荐算法流水线设计上，本章构建了三路召回、粗排、集成精排、重排四层串行管道。召回层以 FAISS 向量检索、ALS 协同过滤、热度兜底三条互补通道并行生成约 600 首候选，而粗排层由 LightGBM 对全量候选快速评分压缩至 300 首，然后由集成层通过两阶段 Stacking 元学习器将 DeepFM 与 BST 两个精排模型的独立评分融合，取前 150 首，最终重排层使用 MMR 算法平衡相关性与多样性，结合冷却过滤后为每位用户生成 50 首个性化推荐。此外在数据库设计上，本章依据 E-R 模型规划了 11 张核心业务表，通过外键约束与联合索引保证数据一致性与查询效率。

AI 62%

在前端交互设计上，本章规划了音乐搜索、个性化推荐、歌单管理、个人中心四个核心功能模块的页面结构，并设计了以播放时长上报、收藏、反馈与屏蔽偏好配置为主的行为采集闭环，使用户的实时交互信号能够持续进入推荐系统的更新链路。以上四个设计域的方案共同构成系统完整的技术架构蓝图，为第五章各核心模块的工程实现提供了明确的设计依据。

5 系统实现

5.1 引言

本章根据第四章系统设计方案，对各个核心模块的实现过程进行了详细的说明。全章按照开发环境配置、数据处理和特征工程、推荐管道算法实现、前端功能实现这四个主线来组织，结合工程实践中遇到的技术关键点以及调优细节，把系统由原始数据到上线服务的所有工作过程展示出来。

5.2 开发环境与工具配置

本系统的开发与训练均在本地单机环境下完成，Python 端承担算法计算与模型训练任务，Java 端承担 Web 服务与页面渲染任务，两端共享同一 MySQL 实例实现数据联动。系统软硬件环境配置如表 5-1 所示。

表5-1 系统实验环境配置

类别	环境	配置
硬件	操作系统	Windows 11 专业版（64 位）
	处理器	13th Gen Intel(R) Core(TM) i9-13900HX
	GPU	NVIDIA GeForce RTX 4060
	显存/内存	8 GB VRAM/16 GB RAM
	CUDA 版本	13.1（驱动 591.74）
Python 端软件	Python	3.12
	PyTorch	2.6.0（cu124）
	LightGBM	4.6.0
	FAISS	1.13.2

表 5-1（续表） 系统实验环境配置

类别	环境	配置
Python 端软件	PySpark	4.0.0
	scikit-learn	1.5.2
	Pandas/NumPy	2.2.2/1.26.4
	Redis	7.4.0
Java 端软件	JDK（Eclipse Adoptium）	23 LTS
	Jakarta Servlet/JSP	6.1/3.1
	MySQL	8.4.0

系统用 C3P0 连接池统一管理数据库连接，启动时预设 5 个连接，在并发峰值时最多可以增加到 20 个，空闲时间大于 300 秒的连接则会自动回收，防止 MySQL 服务端因为长期闲置而引起的连接积压导致超时断开。每次签出连接之前先做存活检查，配合 30 秒的定期心跳检测，保证连接池里的连接一直处在可用状态，最后所有的数据访问方法在异常处理完后归还连接，以确保出现异常不会造成连接泄露。具体的参数配置表 5-2 所示。

表5-2 C3P0连接池配置参数

参数	取值	说明
初始连接数	5	启动时预建连接数
最小连接数	5	连接池保持的下限
最大连接数	20	并发扩展上限
最大空闲时间	300 秒	超时未使用的连接自动回收
获取连接超时	30000 毫秒	超时未获取到连接则抛出异常
空闲连接检测周期	30 秒	定期发送心跳探测连接存活

表 5-2（续表） C3P0 连接池配置参数

参数	取值	说明
签出前存活验证	开启	签出连接前执行轻量查询验活
获取失败重试次数	3 次	连接获取失败后的最大重试数
重试等待间隔	1000 毫秒	每次重试前的等待时间

5.3 数据处理与特征工程实现

5.3.1 数据来源与 ETL 处理

本系统模型训练所用的核心数据来自 KKBOX 公开音乐数据集，是 KDD Cup 推荐系统竞赛数据，包含用户行为交互数据、歌曲元数据和用户画像数据三种结构化数据，一共包含了约 229 万首歌曲、约 737 万条历史收听行为记录和约 3.4 万名注册用户，可以满足大规模推荐算法训练和离线评价的需求。除此之外，从网易云音乐端获得约 2 万首网易云音乐歌曲元数据，经过清洗去重之后和 KKBOX 歌曲数据合并存入歌曲表，一起构成了推荐候选池，大大增加了候选歌曲的覆盖范围。用户行为交互数据有 737 万条样本，每一个样本对应着一次用户和歌曲的交互事件，目标变量是正负样本划分的依据，也是所有的推荐模型训练以及离线评价所需要的基础监督信号，各个字段含义见表 5-3。

表5-3 用户行为交互数据字段说明

字段名称	数据类型	字段说明
msno	String	用户唯一标识
song_id	String	歌曲唯一标识
source_system_tab	String	用户触达该歌曲的功能页面
source_system_tab	String	用户触达该歌曲的功能页面
source_screen_name	String	用户当前所在的具体界面名称

表 5-3（续表） 用户行为交互数据字段说明

字段名称	数据类型	字段说明
source_type	String	播放入口类型
target	Int	目标变量：用户首次收听后一个月 内是否产生重复收听行为（1 为是，0 为否）

AI 56%

歌曲元数据由歌曲基础信息与歌曲扩展信息两部分合并构成，覆盖约 229 万首歌曲，为系统构建歌曲侧内容特征提供原始数据支撑。其国际标准录音制品编码（ISRC）内嵌了发行国家与发行年份信息，通过正则解析可解析出歌曲的来源地区特征与时效性特征，是特征工程阶段歌曲侧特征扩展的重要数据来源，各字段含义如表 5-4 所示。

表5-4 歌曲元数据字段说明

字段名称	数据类型	字段说明
song_id	String	歌曲唯一标识
song_length	Int	歌曲时长
genre_ids	String	流派标识列表
artist_name	String	演唱艺术家名称
composer	String	作曲者名称
lyricist	String	作词者名称
language	Int	歌曲语言编码
name	String	歌曲名称
isrc	String	国际标准录音制品编码

AI 70%
用户画像数据记录了 KKBOX 平台注册用户的基本属性信息，共 3.4 万名用户，用于构建用户侧静态属性特征。其中年龄字段存在大量异常值（如取值为 0 或明显超出合理范围），在后续数据清洗阶段需进行缺失值填充或样本剔除处理，各字段含义如表 5-5 所示。

表5-5 用户画像数据字段说明

字段名称	数据类型	字段说明
msno	String	用户唯一标识
city	Int	城市编码
bd	Int	出生年份
gender	String	性别
registered_via	Int	注册渠道编码
registration_init_time	Int	注册时间戳
expiration_date	Int	会员过期时间戳

ETL 处理阶段用分布式读取的方式，采用分布式计算框架通过标准数据库接口对歌曲信息、播放历史、用户信息、歌单关系这四个主要的业务表进行全量抽取。在执行计划层，框架根据后面特征工程字段依赖关系来自动产生并应用列裁剪（Column Pruning）和谓词下推（Predicate Pushdown）这两种优化策略。其中列裁剪只向数据源请求数据所需要字段，删除与建模无关的冗余字段，而谓词下推把过滤条件提前到数据源端执行，在磁盘读取阶段就丢弃不符合条件的行，而不是在内存里过滤掉。两项优化一起使用，把原来数据在磁盘上读取时的传输量削减约 60%，明显缩减了集群内存的峰值占用情况，加快了数据被分布式内存所占据所需时间，给后面的数据清洗和特征变换工作赋予了高效的运作条件。

5.3.2 数据清洗

原始数据有三类质量问题，需要在特征构建之前一一解决，

（1）缺失值填充

歌曲元数据中 `artist_name`、`genre_ids` 等字段有部分为空值，用语义占位符来代替其中等类型字段，数值型字段（`song_length`）用全局中位数来填补，保留数据行数，不带来极端偏差。训练行为数据里包含少许缺失行，直接舍弃来保证样本标签的完整性。

（2）冷启动过滤

交互次数少于三次的用户由于行为信号非常稀疏，不能对统计特征进行可靠的估计，将这些用户从训练集中去掉。保留至少被播放过一次的所有歌曲，使 `Embedding` 层能够有足够多的样本来学习，防止因为过度过滤而造成召回层候选集太小。

（3）样本分布校验

AI 57% 经对 KKBOX 原始训练数据统计，正样本(`target=1`)约占 50.35%，负样本(`target=0`)约占 49.65%，正负样本天然接近均衡（约 1:1），无需执行额外的下采样操作。清洗阶段只用全量的有效样本参与模型的训练过程，避免由于人为采样造成的分布偏差。

5.3.3 特征工程

AI 55% 系统为粗排层与精排层各模型分别构建了独立的特征集：LightGBM 粗排特征集共 59 维，DeepFM 精排特征集共 50 维，BST 精排特征集共 58。三个特征集在稀疏特征种类与稠密特征组合上各有侧重，但均从用户画像、歌曲属性、行为统计、交叉匹配、时序动态和隐向量嵌入六个维度对"用户-歌曲"交互关系进行刻画，表 5-6 按特征类别对上述特征进行分类展示。

表5-6 特征工程各维度特征概览

特征维度	特征数	代表特征	类型
用户画像	9	性别、年龄、城市、账龄分桶、播放量对数、平均完播	稀疏\
		率、流派多样性熵、近 30 日活跃天数、收听高峰时段	稠密
歌曲属性	11	流派、语言、歌手、来源国、年份分桶、时长分桶、播	稀疏\
		放量对数、平均完播率、归一化热度、歌曲年龄对数	稠密

表5-6(续表) 特征工程各维度特征概览

特征维度	特征数	代表特征	类型
上下文	1	来源渠道 (source_channel)	稀疏
交叉匹配	4	用户-流派匹配度、用户-歌手匹配度、用户-语言匹配度、用户-国家匹配度	稠密
时序行为	8	距上次听同曲天数、历史播放次数、近 7/30 日播放量对数、近 7 日平均完播率、歌曲近 7/30 日播放量、热度趋势比	稠密
SVD 协同嵌入	26	用户-歌曲矩阵分解 10 维、歌曲-用户矩阵分解 10 维、用户-歌手矩阵分解 5 维、SVD 点积分数 1 维	稠密
行为先验	3	用户重复收听率、歌曲重复收听率、用户跳过率	稠密

（1）用户侧特征

用户侧特征分为画像类和行为统计类两种。画像类特征有性别、城市、年龄分段（5 档，未满 18 岁、18~25 岁、26~35 岁、36~50 岁、51 岁以上）、账户成熟度分段（4 档，新用户、成长期、活跃期、老用户）等，用离散化后得到稀疏特征输入嵌入层。

行为统计类特征是在全量历史播放记录基础上预计算出来的，包括用户累计播放总量（经对数变换压缩长尾分布），历史平均完播率，流派偏好多样化的量度用香农熵来量化（熵值越大说明品味越多元），近 30 天活跃天数，历史重复收听率，跳过率（完播进度低于 10%的记录占比较少）以及用户的收听高峰时段。

（2）歌曲侧特征

AI 64%
歌曲侧特征同样分为属性类与统计类两类。属性类特征包括流派标识、发行语言、演唱艺术家、来源国家、年代分段（6 档：1980 年前、20 世纪 80 年代、90 年代、21 世纪 00 年代、10 年代、20 年代）和时长分段（4 档：短曲<90 秒、中等 90~240 秒、较长 240~420 秒、超长>420 秒），以及播放来源渠道。统计类特征包括：全局累计播放总量、全局平均完播率、归一化热度得分、歌曲年龄、歌曲重复收听率和歌曲被

跳过率。

此外引入热度趋势指标以捕捉近期播放动态，其计算方式如式 5-1 所示：

$$\text{trend_ratio} = \frac{c_{7d}+1}{c_{30d}/30+1} \quad (5-1)$$

式中 c_{7d} ——表示该歌曲最近 7 天的播放次数；

c_{30d} ——表示最近 30 天的播放次数。

两者均采用严格的左闭时间窗口计算，以确保特征计算的时间无泄漏性。该指标大于 1 时说明歌曲近期热度加速上升，小于 1 时代表热度趋于衰退。

（3）用户-歌曲交叉特征

系统对流派、艺术家、语言、来源国家四个方面的匹配度进行计算，统计出在用户历史播放记录中，该属性取值相同于候选歌曲的记录所占比例，记为用户对该属性属性匹配度。另外建立时段匹配特征，即当前时段是否在用户历史最高的三个收听时段之内，活跃日匹配特征，即当前日期是否在用户历史上最活跃的前三天之内，从时序上进一步对用户的习惯进行刻画。

（4）时序行为特征

时序行为特征分成记忆衰减特征和滚动窗口统计特征两大类。记忆衰减特性用距离用户上次收听同一种歌曲天数表示（第-1 表示第一次听到这首歌），此时正在播放的这首歌总的收听次数。滚动窗口统计特征用严格的左闭窗口来计算，在 7 天和 30 天这两个时间粒度上对用户播放总量、用户近期平均完播率、歌曲播放总量进行统计，并且将热度趋势指标加入进来体现用户以及歌曲最近的情况。

（5）SVD 隐向量嵌入特征

为将用户-歌曲协同交互信息压缩为低维稠密表示，系统基于截断奇异值分解对用户-歌曲交互矩阵执行矩阵分解，公式见 5-2 所示：

$$R \approx U \Sigma V^T \quad (5-2)$$

式中 R ——为用户-歌曲交互矩阵（发生播放记为 1，否则为 0）；

U ——为用户隐向量矩阵；

Σ ——为奇异值对角矩阵；

V^T 为歌曲隐向量矩阵。

分解后取前 10 个奇异分量分别得到 10 维用户嵌入向量与 10 维歌曲嵌入向量；进一步对用户-艺术家交互矩阵执行同样的分解操作，得到 5 维用户-艺术家嵌入向量；最终以用户向量与歌曲向量的点积作为隐式协同过滤评分。上述嵌入特征共 26 维，作为稠密特征直接拼接至统计特征之中。

(6) OOF 目标编码

高基数稀疏特征（用户标识、歌曲标识等）的类别数目在数万到几十万之间，直接用标签编码的话就会出现目标泄漏的风险，造成训练集评价结果过高。系统采用留出折叠目标编码（Out-of-Fold Target Encoding，OOF-TE）方案，其核心公式见式 5-3 所示：

$$TE_k(x) = \frac{C_{-k(x)} + \alpha \cdot \mu}{N_{-k(x)} + \alpha} \tag{5-3}$$

式中 k ——为当前验证折编号；

$C_{-k(x)}$ ——为除第 k 折外的其余 4 折中特征取值等于 x 的正类样本数；

$N_{-k(x)}$ ——为对应的样本总数；

μ ——为全局正例率；

α ——为平滑系数（本系统取 $\alpha=10$ ）。

如图 5-1 所示，OOF-TE 将训练集划分为 5 折，每一折样本的编码值仅由其余 4 折的统计量计算得出，每条样本的目标编码值均来自其未曾参与训练的数据，从目标上消除了特征计算环节引入的数据泄漏。测试集及线上推理阶段则使用全量训练集统计量计算目标编码，以充分利用所有可用的历史信息。



图5-1 基于折外样本策略的目标编码五折交叉验证机制示意图

5.3.4 数据集时序切分

AI 51%

推荐场景下训练集与验证集的划分必须严格按照时间顺序进行，不能采用随机分组方式，否则验证集中会混入时间上早于训练集的样本，导致模型"提前看到未来数据"，使离线 AUC 虚高而失去参考意义。本系统对不同模型采用对应的时序切分策略：LightGBM 采用用户级时序切分，将每位用户的播放记录按播放时间升序排列后，取各自最后 10%的记录作为验证集，其余作为训练集，交互次数少于 5 次的用户不参与验证集构建；DeepFM 与 BST 则采用全局时序切分，将全体样本按全局播放时间戳升序排列后，统一取最后 10%作为验证集，前 90%进入 K 折 OOF 训练区间，两模型切分基准完全对齐，确保集成阶段 OOF 索引一致。上述时序切分策略模拟了真实推荐场景中"用过去行为预估未来偏好"的评价语义，使验证集 AUC 能够客观反映模型在实际部署环境下的泛化能力。

5.4 MusicMode 推荐算法实现

在完成数据清洗与特征工程基础上，本节依次实现推荐管道的四个核心层次：三路候选召回、LightGBM 粗排、DeepFM 与 BST 双模型精排集成，以及 MMR 多样性

重排，以实现将约 600 首候选歌曲筛选至最终 50 首的推荐结果。

5.4.1 ALS 协同过滤召回实现

AI 64%

ALS 召回模块利用用户与歌曲的历史交互行为构建稀疏交互矩阵并执行矩阵分解，从而为每位用户产生偏好相似的关联候选歌曲集。在实际工程提取中，系统首先从序列化特征内调出用户编码与歌曲编码构建坐标，矩阵内被标识的非零元素代表用户已触发了实际播放行为。此阶段 ALS 推荐模型的隐向量维度设定为 50，收敛迭代轮次设为 10，全局正则化系数配置为 0.1 以应对数据稀疏性。核心训练完成后，系统直接利用每位用户的拟合隐向量与当前全局歌曲隐向量求取内积，将点积得分降序排列后截取最高的 200 首歌曲作为 ALS 召回通道最终的候选分支名单。

5.4.2 FAISS 向量检索召回实现

AI 68%

FAISS 召回通道利用 DeepFM 训练好的 Embedding 层提取歌曲的内容语义向量。具体而言，从 DeepFM 模型中抽取五类稀疏特征的 Embedding：歌曲 ID、流派、语言、歌手、来源国，将五者拼接得到每首歌曲的 160 维（5 类稀疏特征×32 维）复合向量，该向量同时编码了歌曲的协同过滤信息与内容属性信息。

AI 69%

对全量歌曲向量执行 L2 归一化后，以 IndexFlatIP（内积索引）构建 FAISS 索引——L2 归一化后的内积等价于余弦相似度，无需额外计算。用户画像向量由其历史播放歌曲的 Embedding 加权聚合得出（近期播放权重更高），再以该向量在 FAISS 索引中检索，召回前 200 首余弦相似度最高的候选歌曲。整个索引涵盖约 229 万首歌曲，单次检索时延不超过 10ms。

5.4.3 热度兜底召回实现

热度召回通道使用的是预计算过的热度缓存。系统在离线预处理的时候会按照累计播放次数降序排序所有的歌曲，然后取前 10000 首组成热度候选池。在线召回的时候，用每个歌曲的累计播放次数来打基础分数，再加上用户对艺术家的偏爱加权分，系统会根据用户的以往反馈等级，从事先设定好的艺术家偏好的权重映射表里读取相

应的权重分值，对于那些属于偏好的艺术家来说，在候选人当中加上的分是 800 分（用户反馈为不满意）、500 分（中性）、300 分（满意）以及 150 分（非常满意），最后再从中挑选出综合得分最高前 200 首的歌曲来当作热度通道的候选名单。当热度缓存读取失败的时候，系统就自动切换到按照歌曲流行度降序排列的兜底方案上来，保证召回通道的稳定。

5.4.4 LightGBM 粗排实现

粗排阶段用 LightGBM 对三路召回合并后的 600 多首候选歌曲做快速评分，把候选数量减到 300 首以内，然后以直方图优化的梯度提升决策树（Gradient Boosting Decision Tree, GBDT）为基础，对连续特征值进行分桶直方图统计，找到最优分裂点，推理延迟为毫秒级，以实现粗排阶段对于吞吐量要求。

模型输入为 59 维特征，分为用户侧特征、歌曲侧特征和交叉特征。针对训练集中存在的目标泄漏风险，采用 5.3.3 节中所提折外样本目标编码(OOF-TE)方法来构造特征，使得每一个训练样本目标编码值均由其所在折之外的样本统计得到，从而防止标签信息在特征计算时提前泄露。训练使用二分类交叉熵为目标函数，用叶节点最低样本量参数来控制分裂门槛，避免决策树在小样本叶节点上记忆噪声。另外树棵数和学习率的组合控制集成规模和收敛速度，完整超参数配置见表 5-7。

表5-7 LightGBM粗排模型超参数配置

超参数	取值	说明
num_leaves	160	每棵树最大叶节点数
max_depth	6	树最大深度
n_estimators	15000	决策树棵数
learning_rate	0.007	学习率
min_child_samples	2000	叶节点最少样本数，防止过拟合
objective	binary	二分类任务

5.4.5 DeepFM 深度精排实现

AI 54%

DeepFM 模型部署在集成层精排时，为 LightGBM 粗排结果的 300 首候选执行深度建模评分。本模型采用因子分解机(FM)和深度神经网络(DNN)一起训练，在同一个框架里同时学习低阶显式特征交叉和高阶隐式特征交叉，克服了梯度提升树对于高维特征组合建模能力较弱的缺点。

特征输入有稀疏特征和稠密特征两种。14 个稀疏特征用独立 Embedding 表映射到 32 维向量，词表容量另外留一个未见标识符（OOV）的槽位，保证推理阶段出现未知标识符不会发生越界异常。将稀疏特征的 embedding 拼接在一起，得到一个长度为 448 的向量，该向量送入 FM 层以及 DNN 层。FM 层用 $O(kn)$ 的时间复杂度来高效地计算出 91 对二阶特征交叉，接着 DNN 层把所有的稀疏 Embedding 和全部的稠密特征拼接起来，依次经过四层全连接网络，每一层后面都有 ReLU 激活和 Dropout，最后 FM 层和 DNN 层的输出相加后经 Sigmoid 函数映射到(0, 1)区间，得到用户对候选歌曲的偏好概率。

深度模型的输入不含目标编码字段，以免在特征空间里把训练集的标签信息加入进去造成隐性泄露。训练使用全局时序切分，用每位用户最近 10%的交互记录做为验证集来保证评估的时序语义一致，使用混合精度（AMP）训练来降低显存占用，完整超参数配置见表 5-8。

表5-8 DeepFM精排模型超参数配置

超参数	取值	说明
DNN 隐藏层结构	(512, 256, 128, 64)	四层全连接，逐层降维
Dropout 率	0.35	各隐藏层 Dropout，防止过拟合
批量大小	8192	充分利用 GPU 并行计算
最大训练轮次	60	轮次
初始学习率	0.002	Adam 优化器

表 5-8（续表） DeepFM 精排模型超参数配置

超参数	取值	说明
LR 衰减策略	ReduceLROnPlateau	耐心轮次 3，衰减系数 0.35，下限 1e-6
早停耐心轮次	10	AUC 连续 10 轮不提升则停止
混合精度训练	AMP（FP16/FP32）	加速训练，降低显存占用
DNN 隐藏层结构	(512,256,128,64)	四层全连接，逐层降维

5.4.6 BST 行为序列 Transformer 精排实现

BST 核心设计就是使用 Transformer 自注意力机制对用户行为序列做整体建模，用多头自注意力层捕捉序列内各个历史行为之间的时间依赖关系，输出的序列编码向量和候选歌曲特征融合后经过多层感知机输出偏好预测概率，与 DeepFM 的高阶特征交叉视角相补充。

BST 针对稀疏特征做两项针对性的数据预处理。其一是删除频次少的标签词或者用户的频繁标签词，将出现次数少于三次的稀疏标签词都转换成未知标签，防止 Embedding 层因为频繁的高维向量和低频稀疏向量造成模型泛化能力差的问题发生。第二，用用户历史时序位置特征去注入模型，使 BST 对历史序列施加正弦位置编码来捕获序列内各行为的时序顺序关系，让模型可以理解近期的行为和早期的行为，从而消除时序概念漂移的影响。

BST 是用 3 层多头自注意力层和前馈网络堆叠成的 Transformer 编码器，对固定长度为 50 的历史行为序列做全局建模。序列中的每一个行为用 32 维 Embedding 表示，经过线性投影升维到 128 维之后送给 Transformer；编码输出经由注意力加权池化之后和候选歌曲的特征融合在一起，再经过三层全连接网络，分别为 256→128→64，每一层之后都是加入批归一化以及 ReLU 激活之后的输入，输出最终得到的为歌曲的点击概率。完整超参数见表 5-9。

表5-9 BST精排模型超参数配置

超参数	取值	说明
Dropout 率	0.45	Transformer 与 DNN 层共用
批量大小	4096	GPU 显存与吞吐量平衡
稀疏特征数/Embedding 维度	14/32	低频标识符（出现次数<3）映射至未知类别
Transformer 层数/注意力头数	3/4	隐层维度 128，前馈维度 256
DNN 隐藏层结构	(256,128,64)	三层全连接，各层后接批归一化
激活函数	ReLU	各隐藏层后接批归一化
最大训练轮次	40	含早停机制
初始学习率	5e-5	Adam 优化器
LR 衰减策略	CosineAnnealingLR	最小学习率下限 1e-6
早停耐心轮次	7	验证 AUC 连续 7 轮不提升则终止

5.4.7 Stacking 集成与重排输出的实现

（1）Stacking 集成

AI 52%
集成层以 DeepFM 与 BST 两个精排模型在 5 折交叉验证中各自产生的折外(OOF)预测概率为元特征，按全局播放时间戳进行时序切分，训练逻辑回归元学习器拟合最终融合分数，使模型自适应地学习两路精排输出之间的互补关系。元学习器的正则化强度设为中等强度（C=1.0），最大迭代次数设为 1000 轮，以保证在大规模 OOF 样本上收敛。元学习器训练完成后序列化存储至磁盘，在线推荐服务启动时一次性加载至内存，推断阶段不产生额外训练开销。

AI 62%
在线推理阶段，精排集成层将 LightGBM 粗排得到的 300 首候选分别送入 DeepFM 和 BST 两个模型独立评分，随后调用元学习器对两路输出概率组成的特征矩阵执行预测，取正类概率作为最终集成分，从中选出综合得分最靠前的 150 首内容

进入重排阶段。

（2）重排与结果写入

重排阶段用 MMR（最大边际相关）算法来重新排列 150 首候选。算法用贪心的方式选择歌曲，每轮选择 MMR 得分最高的候选加入推荐列表，MMR 得分是考虑候选和用户兴趣的相关性和候选和已选列表最大余弦相似度的权重分别为 0.7 和 0.3 的综合得分。该过程一直进行到选出 50 首为止。

内容过滤在贪心选择的时候同时进行，跳过用户最近已经播放或者显式屏蔽过的项目，对于推荐反馈表中累积冷却惩罚较高的一些内容采取动态降权的方式，在竞争时就自然地往后走。上面的过滤规则都是从数据库实时读取，和集成评分计算在同一个请求上下文中完成。

最后 50 首推荐结果采用批量写入的方式存入到数据库的推荐结果表中，用户发起页面请求的时候直接从数据库中获取并渲染出来，推荐计算和前端服务在时序上完全解耦，不会受到实时推断对接口响应时延的影响。

5.5 MusicWeb 前端实现

5.5.1 MVC 分层架构搭建

系统采用 MVC 架构，在服务端细化为控制层、业务逻辑层、数据访问层、实体层与视图层五个技术层次。控制层由按业务域拆分的多个 Servlet 控制器组成，负责解析 HTTP 请求参数、完成权限校验并调度流程，不直接执行数据库操作，共覆盖用户认证、个人信息管理、音乐服务、行为上报、内容管理、推荐服务、管理端七个功能域；业务逻辑层承载跨数据访问组件的复合操作，推荐服务组件从数据库读取预计算结果并结合收藏与播放状态对推荐列表进行过滤与补全；数据访问层以 C3P0 连接池管理数据库连接，对 11 张核心业务表提供统一的持久化封装，所有 SQL 操作均采用参数化查询规避注入风险，同层的 Redis 工具组件负责热点数据的缓存读写，写操作在更新数据库的同时使相关缓存键失效；实体层由用户、歌曲、歌单、收藏关系、播放历史与申诉等业务实体类构成，作为各层之间传递数据的统一载体；视图层由

JSP 页面、JavaScript 脚本与 CSS 样式表构成，通过异步请求与控制层交互，实现收藏状态变更、推荐列表刷新等无刷新动态操作。

5.5.2 用户认证与账户管理实现

AI 51%

(1) 登录实现：

登录服务端接收用户名与密码参数，完成非空校验后查询数据库获取账号凭据与状态信息，并对三种状态分别处理：状态正常的账号加载完整用户信息，检查并按需补建默认收藏歌单，依据管理员权限标识跳转至用户主页或管理后台，并将用户对象写入服务端会话；冻结账号重定向至状态说明页并附带冻结原因与截止时间；已注销账号重定向至状态说明页并提示相应信息。用户主动退出时，服务端立即销毁会话并跳转至系统首页。登录页面如图 5-2 所示。

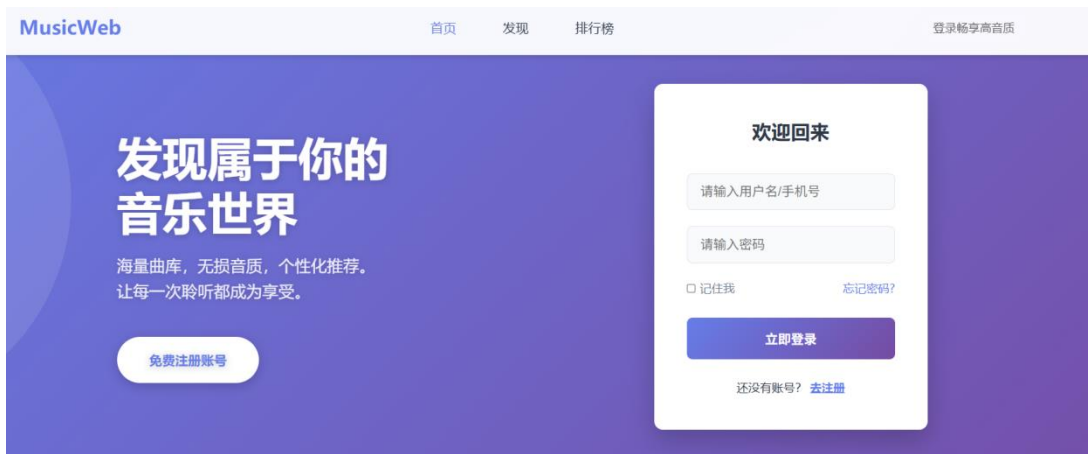


图5-2 系统登录页面

(2) 两步式注册实现：

注册分为两步引导，如图 5-3 所示。第一步是服务端对用户名、密码的非空性以及唯一性进行校验，验证通过后将账号基础信息存入用户表中。第二步服务端接收用户勾选过的流派、歌手喜好信息后存入偏好字段，然后开始推荐引擎冷启动过程生成初始推荐候选集，并创建“我喜欢的音乐”默认收藏歌单；以上操作之后立即开始登录工作，用户不需要手动登录就能直接进入主页。



图5-3 用户注册页面

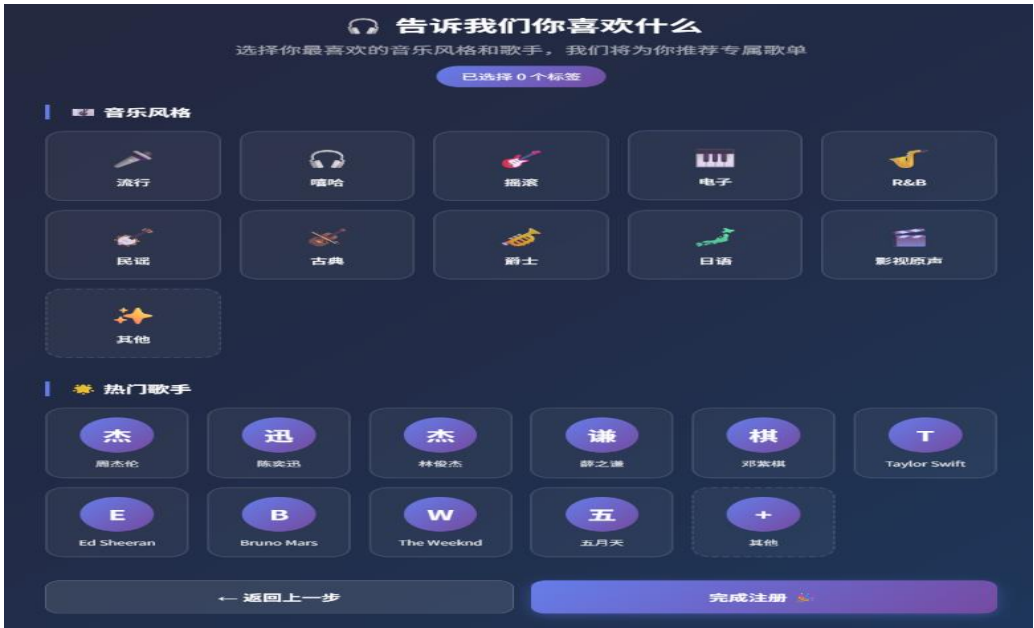


图5-3（续图） 用户注册页面

(3) 基本信息与安全设置

账户设置页面包含个人信息、账户安全、账户管理这三个入口，对应基本资料修改、密码变更、账号注销等三项功能，如图5-4所示。基本信息修改时，服务端对昵称、邮箱、手机号、性别、城市等各字段进行格式校验，即昵称不可超过长度限制，也不能有脚本注入类的特殊字符，邮箱必须符合标准邮件地址格式，以及手机号也要符合

中国大陆11位格式要求，所有校验均通过之后再更新数据库中相应字段并刷新服务端会话。密码修改时，服务端先检验当前密码是否正确，然后对新密码进行强度校验（长度6到20个字节，必须要有字母、数字和特殊字符两种元素之一的）和唯一性检查（不能和目前的密码相同），检验结果通过后更新数据库密码字段和重置会话。账号注销需要输入当前密码进行二次确认，确认无误后服务端对账号进行软删除标记，数据库外键约束自动级联删除相关歌单、收藏记录，然后销毁会话并跳转到首页，密码错误或者系统异常的时候保持会话不变并给出错误提示。

图5-4 账户设置页面

5.5.3 个性化推荐展示

个性化推荐展示界面如图 5-5 所示。推荐数据在 Python 端推荐引擎离线运行之后先存入到数据库推荐结果表中，然后由 Java 端根据需要调用获取并以 JSON 格式发送到前端进行渲染。服务端先检验当前用户的会话是否有效，未登录请求会显示 401 状态码。验证通过后按照综合评分降序读取用户当前推荐列表，每次取 5 首并支持偏移量参数来达到“换一批”的滑动翻页效果，再查询每首歌曲的收藏状态，将歌曲基础信息和收藏状态一起组成 JSON 响应返回给前端。前端推荐卡片展示封面、曲名、演唱者和操作按钮，用户点击播放或者收藏之后，行为数据通过反馈机制回写到推荐反馈表中，供下次推荐计算使用，形成推荐、行为、更新、再推荐的数据闭环。



图5-5 个性化推荐展示页面

5.5.4 音乐搜索与播放收藏实现

搜索服务端根据关键词和来源参数（网易云/QQ/全部）发送请求给外部接口中间服务，结果如图 5-6 所示。对异质响应结构进行解析，把标题、演唱者、封面及时长这些映射到系统内部的歌曲对象上，选择全部来源时采用交替合并方式达到均衡呈现效果。搜索阶段不会进行数据入库操作，当触发播放或者收藏时，服务端对外部歌曲做两阶段去重处理，去重字段为标题、演唱者和演唱者集合的交集，然后依次调用 P1 到 P5 五个降级元数据补全策略，通过网易云百科、QQ 音乐、Last.fm、MusicBrainz、本地检测的方式保证流派和语种字段完整，为推荐特征提取打下基础。

在线播放使用的是 HTML5 原生能力，可以进行进度控制、模式切换。播放触发后开始前端计时器累计时长。切换歌曲或者关闭页面时，异步请求上报标识和实际时长到服务端，服务端根据这两个信息来计算完播率，并且更新推荐反馈记录。执行收藏操作的时候，外部歌曲经过去重入库以及元数据补齐之后，再由服务端联系用户默认歌单并清除缓存。被收藏歌曲若是当日推荐的，则将评分提升，并置位标记，从而形成一个显式正向反馈闭环，进而改善推荐算法准确率。



图5-6 音乐搜索结果页面

5.5.5 歌单管理功能实现

歌单服务端用操作类型参数统一处理创建、添加歌曲、更新、删除四种请求，所有操作都先进行当前会话有效性验证，其效果如图 5-7 所示。创建歌单时，服务端使用用户标识以及自定义的名字来创建一个新的歌单记录，将其标记为非默认歌单。当目标歌曲来源于外部平台时，服务端会先进行和 5.5.4 节一样的两阶段去重入库操作，然后检查歌单归属权，符合条件的就将歌曲加入到该歌单中。在修改歌单名称或者描述的时候也要进行归属权的检查，在删除操作时又会拦下对于默认歌单（"我喜欢的音乐"）的删除请求，避免因误删而造成收藏容器的删除。全部写入操作都会用到 JSON 格式返回操作结果。



图5-7 用户歌单管理页面

5.5.6 管理员后台功能实现

管理员后台以统一的服务入口接收操作类型参数，将请求路由至系统统计、用户信息管理、申诉管理、歌单管理、歌曲管理五个功能模块，所有请求均先校验会话中的管理员身份标识，非管理员请求直接拦截。

（1）系统统计

统计模块聚合数据库中的用户总量、歌曲总量、播放记录数与推荐条目数四项关键指标，展示于数据看板，同时在后台静默执行保护期满 30 天的注销账号物理清理，如图 5-8 所示。

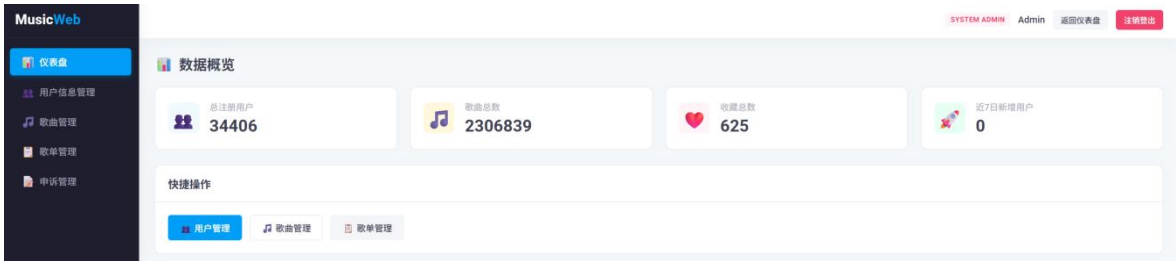


图5-8 管理员后台数据统计看板

（2）用户信息管理

AI 67%
用户信息管理模块支持分页查询全部用户列表与单用户详情。账号冻结操作将账

号状态更新为冻结并记录冻结原因与截止日期，而解冻操作则将状态恢复为正常。此外软删除操作将账号标记为已注销并设置 30 天数据保护期，期内数据完整保留。用户申诉审核通过后由申诉模块自动触发解冻或账号恢复，用户信息管理页面如图 5-9 示。



图5-9 管理员后台用户信息管理页面



图5-9（续图） 管理员后台用户详情页面

(3) 歌曲管理

歌曲管理模块支持关键词分页搜索歌曲列表，可对歌曲执行添加、删除与编辑操作；编辑保存后自动清除服务端歌曲信息缓存，确保前端数据一致性。管理员还可在列表中直接预览播放歌曲，验证音频文件可用性，如图 5-10 所示。



图5-10 管理员后台歌曲管理页面

(4) 歌单管理

AI 69%
歌单管理模块以全局视角展示系统内所有用户的歌单列表，每条记录附带创建者信息与歌曲数量。管理员可查看任意歌单的歌曲明细，对违规歌单执行删除操作或从歌单中移除特定歌曲；系统对默认歌单（"我喜欢的音乐"）设有保护，禁止管理端强制删除用户的收藏容器，如图 5-11 所示。

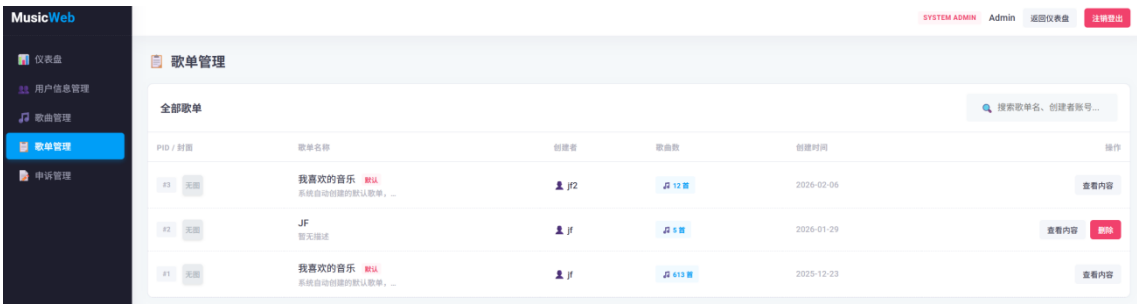


图5-11 管理员后台歌单管理页面

(5) 申诉管理

AI 69%
申诉管理模块展示全部用户提交的申诉列表及详情，含申诉类型、原因与联系邮箱。批准申诉时，服务端修改申诉状态、解除账号冻结或注销标记，并通过异步线程向用户联系邮箱发送审核通过通知。而拒绝申诉时则同步更新状态并发送拒绝通知邮件，如图 5-12 所示。

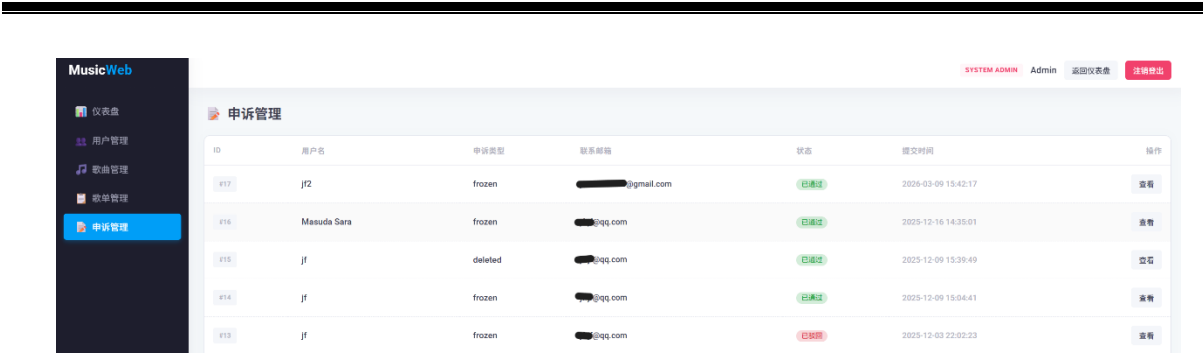


图5-12 管理员后台申诉管理页面

5.5.7 隐式反馈采集实现

隐性行为侦测和显式意图采集形成两个不同的途径来实现用户反馈，二者一起推动着推荐引擎不断升级。隐式通路中，用户触发播放之后，前端计时器立刻开始记录有效的收听时长，当切歌、关页或者自然结束任何一个事件发生的时候，前端就以异步请求的方式把歌曲编号和实际时长一起发给服务端，服务端计算完播率之后会写入推荐反馈表，并且更新播放状态。完播率小于 20%的行为被认定为跳曲，系统会对该歌曲以及艺人的累积冷却分值进行惩罚，在后续的候选排序竞争中使其自然靠后，从而减少对用户明确抗拒内容的重复展示。在显式通路里，用户可以借助口味反馈页面对当日推荐歌曲的满意度进行打分，服务端会把这些数据写入到口味反馈表当中，经过去重之后再一起写入到用户信息表中并刷新会话里的偏好缓存，使得之后的推荐计算使用到的都是最新的偏好设定。

5.6 本章小结

AI 70%
本章系统阐述了推荐系统从底层数据流转到业务应用落地的完整工程实现过程。为保障高并发访问的稳定性，系统预先对 C3P0 连接池的核心参数及心跳探活机制进行了专项调优。在数据处理层面，依托 PySpark 分布式计算框架执行了 KKBOX 全量数据集的 ETL 流，并结合列裁剪与谓词下推技术，将磁盘 I/O 开销进行大幅削减。数据清洗工作依次排除了缺失值与冷启动用户的干扰，并严格校验了正负样本分布。

AI 96%
在特征工程环节，本研究依据不同算法的数据偏好，分别为 LightGBM、DeepFM

与 BST 模型构建了 59 维、50 维及 58 维的特征子集，全面覆盖了用户画像、时序行为与 SVD 隐向量等六大维度。通过引入折外目标编码（OOF-TE），有效阻断了高基数稀疏特征可能引发的目标泄漏风险。同时，模型切分采用用户级别与全局时序末尾 10%相结合的对齐策略，最大程度还原了真实环境的推荐语境。

AI 81%

推荐管道的实现构成了系统的核心架构。召回层混合了 ALS 协同过滤、FAISS 向量检索与热度兜底策略，初步获取约 600 首候选曲，然后 LightGBM 粗排层将其筛选至 300 首。再由精排阶段采用双模型并行架构进行打分：DeepFM 主攻高阶交叉特征挖掘，BST 专注用户行为序列建模。这两路评分统一汇入以逻辑回归为元学习器的 Stacking 集成层进行权重融合，截取最优的 150 首曲目。最终，重排层调用 MMR 算法平衡结果的相关性与多样性，输出 50 首最终推荐清单并完成落库。

AI 70%

在前端交互层，MusicWeb 系统基于标准 MVC 架构完成了多源音乐检索、歌单管理等核心业务开发。系统依托 HTML5 播放器的完播率隐式埋点机制，配合用户显式评分双轨并行，打通了数据反馈闭环，确保推荐列表的动态演进。至此，本系统的主体工程已全线贯通。后续第六章将结合离线量化指标与在线评估指标，对推荐体系的实际应用效能展开全面验证。

6 系统测试与评估

6.1 引言

AI 69%
第五章完成了四层推荐管道与 MusicWeb 前端平台的全量工程实现，系统已具备向真实用户提供个性化推荐服务的能力。但工程实现是否真正有效，需要以量化指标加以验证。为此，本章以 KKBOX 数据集为基准，从纵横两个维度展开离线评估：纵向通过逐层消融实验，逐步叠加召回、粗排、精排集成与重排四个层级，量化各模块对排序质量的实际贡献；横向则在同一验证集上对单体模型与集成框架的点击率判别能力进行比较，考察 Stacking 架构相对于独立模型的优势所在。

6.2 评估指标与实验设置

6.2.1 评估指标说明

AI 67%
离线评估选取命中率（HR@K）、精确率（Precision@K）、归一化折损累积增益（NDCG@K）、平均倒数排名（MRR）列表内多样性（Shannon 熵）与曲下面积（AUC）六项指标，从不同角度量化推荐结果的排序质量与覆盖多样性，前四项均在 Top-10 推荐场景下计算（K=10）。

(1)命中率

命中率衡量系统能否将用户真正感兴趣的内容纳入推荐列表前 K 位，即验证集中至少命中一个正样本的用户占比，命中率越高说明系统对绝大多数用户均能在有限推荐位中呈现其偏好内容，公式见式（6-1）所示。

$$HR@K = \frac{1}{|U|} \sum_{u \in U} 1[rank_u \leq K] \quad (6-1)$$

式中 U——为验证集用户集合；

|U|——为用户总数；

rank_u——为用户 u 的真实正样本在推荐列表中的排名位置；

1[·]为示性函数，当括号内条件成立时取值为 1，否则为 0。

(2)精确率

精确率衡量推荐列表中相关内容所占的比例，精确率越高说明系统推荐的内容中
有用信息密度越大，用户无需在大量无关推荐中筛选目标，在音乐推荐场景下直接关
联用户对推荐列表的整体满意度，公式见式（6-2）所示。

$$\text{Precision@K} = \frac{1}{|U|} \sum_{u \in U} \frac{R_u \cap T_u}{K} \quad (6-2)$$

式中 R_u ——为系统为用户 u 生成的 Top-K 推荐列表；

T_u ——为用户 u 的真实正样本集合；

K ——推荐列表长度，本系统取 $K=10$ 。

(3)归一化折损累积增益

AI 66%

归一化折损累积增益在命中基础上引入排名位置的对数折损权重，对排名靠前的
命中给予更高奖励，能够区分"排在第 1 位命中"与"排在第 10 位命中"的质量差异，
更精细地反映系统将偏好内容优先排列于列表前端的能力，计算公式见式（6-3）和
（6-4）所示。

$$\text{NDCG@K} = \frac{1}{|U|} \sum_{u \in U} \frac{DCG_u}{IDCG_u} \quad (6-3)$$

$$DCG_u = \sum_{i=1}^K \frac{rel_i}{\log_2(i+1)} \quad (6-4)$$

式中 $IDCG_u$ ——为用户 u 在理想排序下的最大 DCG 值，即将该用户所有正样本置
于列表最前端时所能达到的 DCG 上界；

DCG_u ——用户 u 的折损累积增益；

rel_i ——第 i 个推荐位置的相关性标签，命中正样本时取值为 1，否则为 0；

$\log_2(i+1)$ ——位置折损因子，使排名越靠后的命中贡献以对数速率递减；

K ——推荐列表长度，本系统取 $K=10$ ；

(4) 平均倒数排名

平均倒数排名以每位用户正样本首次出现位置的倒数取均值，对列表头部极为敏
感，正样本出现在第 1 位贡献为 1.0，出现在第 5 位仅为 0.2，MRR 高的系统不仅能

命中用户兴趣，还能将最相关内容优先呈现，公式见式（6-5）所示。

$$MRR = \frac{1}{|U|} \sum_{u \in U} \frac{1}{rank_u} \quad (6-5)$$

式中 $rank_u$ ——为用户 u 的正样本在推荐列表中首次出现的排名位置；

$|U|$ ——为验证集用户总数。

（5）列表内多样性

列表内多样性以推荐列表中流派分布的 Shannon 熵为度量，熵值越高说明推荐结果覆盖的音乐风格越丰富，用户不会陷入单一流派的信息茧房。对每位用户的 Top-K 推荐列表统计各流派占比后计算熵值，再对所有用户取均值，公式见式（6-6）所示。

$$H = - \sum_{c \in C} p_c \log_2 p_c \quad (6-6)$$

式中 H ——单用户推荐列表的流派 Shannon 熵；

C ——推荐列表中出现的流派集合；

P_c ——流派 c 的歌曲数占推荐列表总长度的比例；

$\log_2$ ——以 2 为底的对数，使熵值单位为比特。

（6）曲线下面积

AUC（Area Under the ROC Curve）为 ROC 曲线下面积，衡量模型区分正负样本的全局排序能力。其等价含义为：从正样本集与负样本集中各随机取一个样本，模型将正样本得分排于负样本之上的概率，AUC 越高说明模型的二分类判别能力越强，计算公式见式（6-7）所示。

$$AUC = \frac{1}{|P| \cdot |N|} \sum_{p \in P} \sum_{n \in N} 1[f(p) > f(n)] \quad (6-7)$$

式中 P ——正样本集合；

N ——负样本集合， $|P|$ 与 $|N|$ 分别为对应集合的样本数量；

$f(\cdot)$ ——模型对样本的预测得分函数；

$1[\cdot]$ ——示性函数，当括号内条件成立时取值为 1，否则为 0。

6.2.2 实验设置

AI 68%

本文设置两组实验，使用 KKBOX 数据集，涵盖 28172 名用户约 73 万条交互记录，按用户和全局时序划分，取每位用户最后 10% 的交互数据作为验证正样本，其余数据用于训练。

针对多层级推荐架构，设计了逐级递进的消融实验，重点量化拆解管道内各个功能模块的实际增益。该测试通过四个评估节点来追踪性能演进，首先基础对照组仅保留召回环节，由三通道 ALS 并联产出约 600 首候选曲目，并以均匀随机打分作为推荐下界。随后，系统接入 LightGBM 粗排层，观察候选规模被强过滤至 Top-100 时的指标变动。在精排干预节点中，引入了 Meta-LR 集成模块，利用元学习器对 DeepFM 与 BST 的双路输出进行融合，完成对这 600 首候选的二次深度评分。最终环节则部署了 MMR 重排机制，以实现多样化输出。在这一递进测试链路中，主要依靠 HR@10、Prec@10、NDCG@10 及 MRR 四项核心指标来衡量排序质量的优劣，而针对完成多样性后处理的 MMR 阶段，还额外增加了推荐列表的流派 Shannon 熵，用于客观量化系统在曲风丰富度上的改善。

AI 70%

除了纵向拆解还设置了横向对比实验，其聚焦于单体模型与集成框架的预估准度差异。在该阶段，LightGBM 粗排模型、独立的 DeepFM 与 BST 模型，以及 Meta-LR 集成架构在同一验证集下展开点击率判别能力的测试，所有参评对象均以 AUC 得分作为唯一量化准绳。值得说明的是，由于 ALS 协同过滤与热度召回策略产出的是非概率性质的相对分值，无法直接绘制 ROC 曲线，因此这两类基线算法仅在前置的消融测试中参与排序指标评估，被排除在 AUC 对比之外。此外，为彻底规避过拟合与标签泄露隐患，Meta-LR 所输出的预测概率严格取自五折交叉架构下的折外预测（OOF）得分。这种处理方式能够更真实地映射模型面对未知样本的泛化水平，从而为最终的 AUC 对比数据提供了可靠的逻辑支撑。

6.3 消融实验结果分析

本实验目的是验证四层推荐管道中各环节的实际作用，分别对召回层、粗排层、精排集成层、重排层使用 6.2.1 节的指标进行验证，实验结果如图 6-1 所示。

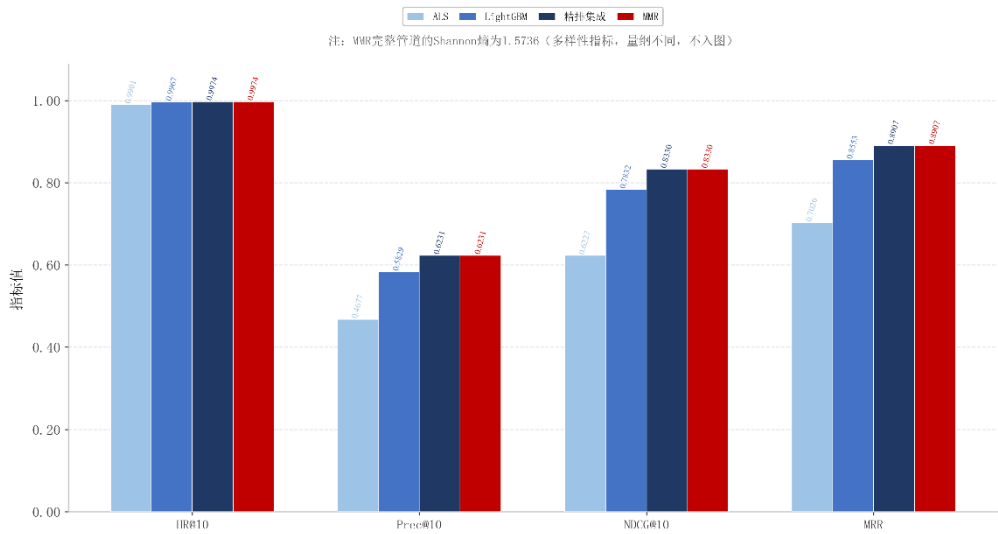


图6-1 四层管道指标得分柱状图

从图中可以看出，四个配置的差异主要集中在 $Prec@10$ 、 $NDCG@10$ 和 MRR 三项指标上。仅召回层输出时 $HR@10$ 已达 0.9901，说明 ALS 三通道并联生成的约 600 首候选对绝大多数用户均能覆盖至少一首偏好歌曲，后续各层的主要作用在于精化排序质量而非扩大命中覆盖。叠加 LightGBM 粗排后， $Prec@10$ 从 0.4677 提升至 0.5829， $NDCG@10$ 从 0.6227 提升至 0.7832， MRR 从 0.7026 提升至 0.8553，三项指标在四个叠加阶段中的提升幅度均为最大，该结果与粗排的工程定位相符。LightGBM 将约 600 首召回候选压缩至 Top-100，候选集中低相关样本的占比降低，进入推荐列表的正样本密度随之提高。在粗排基础上进一步叠加精排集成后， $Prec@10$ 、 $NDCG@10$ 、 MRR 分别提升至 0.6231、0.8330、0.8907，Meta-LR 元学习器融合 DeepFM 的特征交叉能力与 BST 的序列感知能力，推动正样本向列表头部集中， MRR 为 0.8907 意味着正样本平均首次出现位置已接近第 1.1 位。在叠加 MMR 重排后构成完整推荐管道，而四项配对级指标与精排集成配置完全一致，是因为多样性贡献体现在推荐列表的流派 Shannon 熵上——均值为 1.5736 比特，表明 MMR 重排在不损失排序精度的前提下拓宽了推荐结果的流派覆盖范围。综合各阶段指标变化可以看出，粗排对精度类指标的边际贡献最为突出，精排集成进一步提升了排序质量，MMR 重排在精度保持不变的情况下补充了多样性，四层管道各环节均有实质性贡献，叠加路径上的指标变化与设计预期一致。

6.4 各模型 AUC 对比实验

AI 67%

本实验旨在横向评估各模型在验证集上的点击率判别能力，参与对比的模型包括 LightGBM 粗排模型、单模型 DeepFM、单模型 BST 与集成模型（Meta-LR），以 AUC 为统一评估指标，结果如图 6-2 所示。

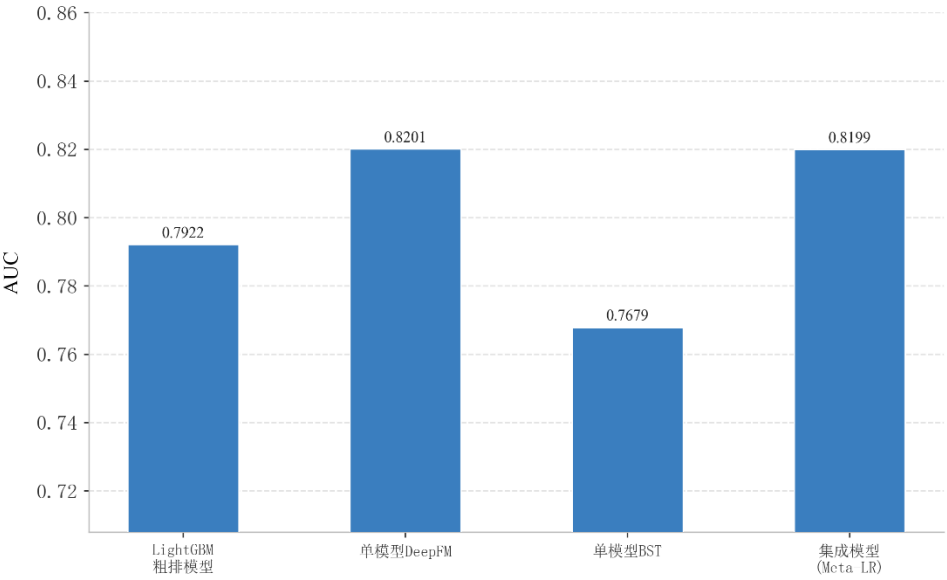


图6-2 各模型AUC得分

AI 63%

从图中可以看出各模型排名与其建模方式有较强的对应关系。对于最高的 AUC 得分模型 DeepFM，其因子分解机与深度网络并联的结构天然适合静态特征间的高阶交叉，这一优势使其在 KKBOX 特征结构下得到充分体现。而 AUC 为 0.7922 的 LightGBM 模型，其 59 维输入特征涵盖用户行为统计、热度衰减、流派偏好等聚合信息，因此该显式特征编码为粗排判别提供了扎实基础。对于四个模型中最低的 BST 模型而言，其原因并不在于 Transformer 序列建模的设计本身，而在于 KKBOX 中部分用户历史记录稀疏。其自注意力机制在序列不足的条件下难以充分学习行为规律，因此序列建模的优势大打折扣。相较于 DeepFM 模型，集成模型的 AUC 基本与其持平，两者仅差 0.0002，低于 0.001 的统计显著性阈值，其判别能力实质上与 DeepFM 持平。

AI 66%

除此之外，AUC 度量的是全量样本上的二分类判别能力，并不直接等同于推荐列表的排序质量，需结合 6.3 节消融实验加以说明。可以看到粗排在叠加精排集成后，其 Prec@10 从 0.5829 提升至 0.6231，NDCG@10 从 0.7832 提升至 0.8330，MRR

从 0.8553 提升至 0.8907，三项排序质量指标均有改善。这说明集成模型通过融合 DeepFM 对稀疏特征间隐式交叉关系的自动建模能力与 BST 对用户历史行为序列中动态偏好信号的提取能力，在正样本的精细排序上弥补了 LightGBM 仅依赖人工构建特征组合、无法感知行为时序依赖的局限。

AI 70%

综上所述，各模型在实际推荐链路中均有发挥一定的作用。LightGBM 在承担粗排阶段的初步过滤任务表现符合预期，DeepFM 在处理静态特征交叉时的单点排序能力最为突出，而 BST 由于受制于 KKBOX 数据集的重度稀疏性，未能充分释放其序列建模的理论潜力。这也是本研究引入集成层设计的初衷，其核心诉求并非单纯追求全量样本下 AUC 绝对值的微弱攀升，而是试图将 DeepFM 的静态交叉视界与 BST 的时序动态信号进行优势互补。从前文消融实验中各项排序质量指标的实际涨幅来看，这种多视角特征融合策略切实优化了列表的头部推荐精度，进而从数据层面支撑了本研究多层次推荐管道架构的合理性。

6.5 本章小结

AI 69%

依托 KKBOX 数据集，本章围绕四层推荐架构实施了两组离线验证。测试前明确了数据时序切分策略，并引入 HR@10、NDCG@10、AUC 等六项核心指标，构筑了统一的量化对比基准。

消融实验通过四个递进配置逐层验证了管道各模块的实际贡献。召回层的 HR@10 达到 0.9901，说明三通道并联召回对绝大多数用户的覆盖作用起效，而随着粗排与精排集成的逐步叠加，Precision@10、NDCG@10 与 MRR 也持续上升，排序质量得到稳步改善，以及重排引入 MMR 机制后 Shannon 熵达到 1.5736，证明各层均对最终推荐质量产生了可量化的正向增益。

AI 70%

模型对比实验中，DeepFM 的 AUC 得分居四个模型之首，其因子分解机与深度网络并联的结构在静态特征交叉上具备优势，而 LightGBM 则稳定承担粗排筛选任务，整体表现符合预期。BST 受限于 KKBOX 数据集的高稀疏性，序列建模能力未能充分释放。集成学习在 OOF 框架下融合 DeepFM 与 BST 的互补视角，三项排序质量

指标均优于任一独立模型，验证了 Stacking 架构的合理性。

结 论

本文根据 KKBOX 公开数据集，设计出了一个较为完善的个性化音乐推荐系统，并且采用三路召回、粗排、精排集成、重排四个阶段，通过 Stacking 集成验证集 AUC 达到 0.8199，基本达到预期工程目标。

（1）主要工作总结

AI 70%

对于如何实现千人千面个性化推荐的目标，本文从数据、算法与交互三个维度展开论述。数据层面，以 KKBOX 数据集为基础构建 ETL 管道，结合 OOF 目标编码策略消除高基数类别特征的标签泄漏风险，为每位用户提炼出涵盖行为统计、流派偏好与时序趋势的个人画像特征，使模型能够捕捉用户偏好的细粒度差异。算法层面，四层管道针对不同用户状态提供差异化的推荐路径，三通道并联召回保障活跃用户与冷启动用户均能获得有效候选，LightGBM 粗排完成效率优先的初步过滤，DeepFM 与 BST 分别从静态特征交叉和行为序列时序建模两个互补视角精细评估用户兴趣，经 Meta-LR 融合后再由 MMR 重排主动调节流派多样性，避免推荐结果趋于同质。交互层面，MusicWeb 前端通过节流机制异步采集用户播放完成率与评分行为，持续回写推荐系统，构建“推荐—反馈—优化”的闭环，使用户画像随使用深度不断丰富。

（2）创新点

AI 97%

本文核心改进贯穿特征流转、模型架构与输出干预三个节点。面对高基数类别特征易引发标签穿越的问题，在特征构建环节引入折外目标编码（OOF-TE）。在交叉验证框架下，当前折编码仅利用外部样本统计量，通过底层数据的物理隔离切断标签渗漏，夯实了评估的客观性。在确保特征纯净后，推荐链路实施了分层解耦的异构集成架构。前置阶段由 LightGBM 依托 59 维特征承接大规模候选降维以保障吞吐率；精排节点则部署 DeepFM 与 BST 双轨并行网络，促使静态高阶交叉与动态时序依赖优势互补，最后交由 Meta-LR 执行权重融合。此架构将多视角建模纳入同一链路，使集成模型取得了反超单一基线的排序精度。此外，为避免单纯优化点击率加剧流派同质化，输出末端追加最大边际相关（MMR）重排机制作为干预手段，对曲风分布施

加软性约束，在维持头部准确率的同时有效瓦解信息茧房，实现精度与广度的综合平衡。

（3）不足与展望

本研究存在以下几点不足：系统部署于单机环境，缺乏生产级别的高可用保障；BST 的序列建模能力受制于 KKBOX 数据集的高稀疏性，Transformer 的长程依赖优势未能充分发挥；冷启动用户的推荐仍依赖注册偏好与热度兜底，画像精细度与活跃用户相比尚有差距；离线指标与真实用户体验之间的映射关系缺乏大规模在线实验的支撑。

后续可从以下方向推进：引入图神经网络对用户-歌曲交互关系建模，或以对比学习方法缓解数据稀疏问题。其次是将系统迁移至微服务与容器化架构，结合实时特征流实现推荐的动态更新，亦或是建立在线 A/B 测试框架，以点击率、留存率等真实业务指标对离线评估体系加以验证与补充。

参考文献

- [1] 赵吉.基于 Spark 的个性化音乐推荐系统设计与实现 [D].长春: 吉林大学, 2022.
- [2] 杨率.基于深度神经网络的个性化实时音乐推荐系统研究与实现 [D].北京: 北京邮电大学, 2022.
- [3] 李津.基于知识图谱的个性化音乐推荐系统设计与实现 [D].北京: 北京交通大学, 2023.
- [4] 冯亚寒.基于 WSM-CNN 及其改进的音乐多情感分类方法研究与应用 [D].重庆: 西南大学, 2023.
- [5] 吴亚迪, 陈平华.基于用户长短期偏好和音乐情感注意力的音乐推荐模型 [J].广东工业大学学报, 2023, 40 (04): 37-44.
- [6] 王慧心, 童向荣.融合知识图谱的推荐系统研究进展 [J].浙江大学学报 (工学版), 2023, 57 (08): 1527-1540.
- [7] 梁锋, 羊恩跃, 潘微科, 等.基于联邦学习的推荐系统综述 [J].中国科学: 信息科学, 2022, 52 (05): 713-741.
- [8] 郑楠, 章颂, 刘玉桥, 等.基于深度学习的群组推荐方法研究综述 [J].自动化学报, 2024, 50 (12): 2301-2324.
- [9] 闫猛猛, 汪海涛, 贺建峰, 等.基于自监督学习的序列推荐算法 [J].重庆邮电大学学报 (自然科学版), 2023, 35 (04): 722-731.
- [10] 龚雪鸾, 陈艳姣, 王帅.在线广告点击率预测方法的研究综述 [J].中文信息学报, 2023, 37 (04): 1-17.
- [11] Liu Z, Xu W, Zhang W, et al. An emotion-based personalized music recommendation framework for emotion improvement [J]. Information Processing & Management, 2023, 60(3): 103256.
- [12] Wang D, Zhang X, Yin Y, et al. Multi-view enhanced graph attention network for session-based music recommendation [J]. ACM Transactions on Information

- Systems, 2023, 42(1): 16.
- [13] Li L, Tao D, Zheng C, et al. Attentive autoencoder for content-aware music recommendation [J]. CCF Transactions on Pervasive Computing and Interaction, 2022, 4(1): 76-87.
- [14] Mao Y, Zhong G, Wang H, et al. Music-CRN: An efficient content-based music classification and recommendation network[J]. Cognitive Computation, 2022, 14(6): 2306-2316.
- [15] Paul S, Singh S, Rajbhoj S. Personalized music recommendation system [EB/OL]. (2021-01) [2026-03-18]. <https://ssrn.com/abstract=3772631>.
- [16] Niu X, Wang J, Chen X. Collaborative filtering-based music recommendation in Spark [J]. Mathematical Problems in Engineering, 2022: 1-12.
- [17] Johnson J, Douze M, Jegou H. Billion-scale similarity search with GPUs [J]. IEEE Transactions on Big Data, 2021, 7(3): 535-547.
- [18] Wang X, Li Q, Yu D, et al. Neural causal graph collaborative filtering[J]. Information Sciences, 2024, 680: 120872.
- [19] TRAN V A, SALHA-GALVAN G, HENNEQUIN R, et al. Transformers meet ACT-R: repeat-aware and sequential listening session recommendation[C]// Proceedings of the 18th ACM Conference on Recommender Systems. New York: ACM, 2024: 486-496.
- [20] CHEN Q, ZHAO H, LI W, et al. Behavior sequence transformer for e-commerce recommendation in Alibaba[C]// Proceedings of the 1st International Workshop on Deep Learning Practice for High-Dimensional Sparse Data. New York: ACM, 2019: 1-4.

致 谢

AI 70%

总觉来日方长，毕业无期，行文至此，也百感交集。

人生中有许多节点，有些走得轻盈，有些却走得深重。这两年，无疑是我求学轨迹里权重最大的一段。从 2025 年冬日敲下第一行字，到如今数月间的字斟句酌、反复推敲，其间的甘苦，犹如一次漫长且充满未知的模型训练。二十四岁，不算年轻，却也不算老，刚好是一个人应当学着对自身命运按下“回车键”的年纪。这篇论文，便是我在这个路口，交出的一份最为诚恳的答卷。

AI 70%

最深的谢意，首先要献给我的指导老师王波老师。在这场漫长的学术马拉松里，无论是线上的探讨还是线下的指点，王老师从未缺席。从最初选题时帮我锚定方向，到系统框架构建中的反复斟酌，再到初稿里那些我自以为严密、实则逻辑断层的段落，王老师的每一次批注都能直击要害，绝不以空洞的套话敷衍。学问之道，贵在真诚与严谨。正是王老师那如同底层代码般扎实而耐心的引导，才让这篇论文得以拨开冗杂的噪点，一步步走向清晰。师恩如海，我将永远铭记。

AI 70%

感谢广州华立学院计算机工程学院的每一位任课老师。两年的光阴里，算法、数据库、分布式计算、机器学习……这些名词在黑板上出现时，我未必真切地感受到它们的重量。然而，当我真正坐在屏幕前，开始搭建这套个性化音乐推荐系统时，我才陡然发觉，那些曾经以为枯燥的知识，早已悄无声息地沉入地基，支撑起了整个系统的每一处细节。知识的给予往往是无声的，却总能在你最需要的时候，化作最坚实的算力。

AI 70%

最深沉的感激，要留给我的家人。他们或许永远弄不懂什么是协同过滤，什么是推荐算法，也看不明白终端里跳动的代码，但这从未妨碍他们给予我百分之百的爱。十七年的求学路，每一次跌倒与迷茫，都有他们在身后稳稳托底。他们不苛求我取得多耀眼的成绩，只求我平安顺遂。这份爱，不张扬、不喧哗，却是我人生长河里最不可或缺的前行动力，也是我越过所有难关时最踏实的依靠。

AI 70%

行文至此，这段学生时代的故事即将画上句点。但这并非终局，只不过是跳出了当前的循环，准备去往一个更广阔的天地重新运行。二十四岁，往后的路还长，愿我能走得踏实，走得辽阔，带着这十七年积攒的所有勇气，去迎接真实世界里更为庞杂的数据洪流。

欲买桂花同载酒，终不似，少年游。谨以此篇，献给所有陪我走过这段时光的人。

附 录

附件 1:

表 4-1 用户表结构

字段名	数据类型	约束	说明
id	INT	PK,AUTO_INCREMENT	用户主键
username	VARCHAR(50)	UNIQUE NOT NULL	用户名
password	VARCHAR(255)	NOT NULL	密码
email	VARCHAR(100)	—	邮箱
nickname	VARCHAR(50)	—	昵称
phone	VARCHAR(20)	—	手机号
status	ENUM	NOT NULL	active/frozen/deleted
frozen_until	TIMESTAMP	—	冻结截止时间
frozen_reason	VARCHAR(255)	—	冻结原因
deleted_at	TIMESTAMP	—	逻辑删除时间
preferred_genres	VARCHAR(255)	—	偏好流派
preferred_artists	VARCHAR(255)	—	偏好艺术家
create_time	TIMESTAMP	DEFAULT NOW()	注册时间

表 4-2 歌曲表结构

字段名	数据类型	约束	说明
id	INT	PK,AUTO_INCREMENT	歌曲主键
title	VARCHAR(255)	NOT NULL	歌名
artist	VARCHAR(255)	—	歌手
album	VARCHAR(255)	—	专辑
duration	INT	—	时长（秒）
genre	VARCHAR(100)	—	流派
genre_ids	VARCHAR(255)	—	KKBOX 原始流派 ID 列表

广州华立学院本科生毕业论文（设计）

字段名	数据类型	约束	说明
language	VARCHAR(10)	—	语言代码
release_year	INT	—	发行年份
kkbox_id	VARCHAR(100)	—	KKBOX 原曲 ID
popularity	INT	DEFAULT 0	歌曲热度
origin_country	VARCHAR(50)	—	原产地
cover_image	VARCHAR(500)	—	封面图片路径

表 4-3 播放历史表结构

字段名	数据类型	约束	说明
id	INT	PK,AUTO_INCREMENT	主键
user_id	INT	FK→users.id NOT NULL	用户 ID
song_id	INT	FK→songs.id NOTNULL	歌曲 ID
play_time	TIMESTAMP	NOT NULL	播放时间

表 4-4 用户歌单表结构

字段名	数据类型	约束	说明
id	INT	PK,AUTO_INCREMENT	主键
user_id	INT	FK→users.id NOT NULL	创建者 ID
name	VARCHAR(100)	NOT NULL	歌单名称
description	TEXT	—	歌单描述
cover_image	VARCHAR(500)	—	歌单封面
is_default	TINYINT(1)	DEFAULT 0	是否为默认收藏夹
create_time	TIMESTAMP	DEFAULT NOW()	创建时间
update_time	TIMESTAMP	DEFAULT NOW()	最后更新时间

表 4-5 歌单歌曲关联表结构

字段名	数据类型	约束	说明
id	INT	PK,AUTO_INCREMENT	主键
playlist_id	INT	FK→user_playlists.id NOT NULL	歌单 ID
song_id	INT	FK→songs.id NOT NULL	歌曲 ID
add_time	TIMESTAMP	DEFAULT NOW()	添加时间

表 4-6 推荐结果表结构

字段名	数据类型	约束	说明
id	INT	PK, AUTO_INCREMENT	主键
user_id	INT	FK→users.id NOT NULL	目标用户 ID
song_id	INT	FK→songs.id NOT NULL	推荐歌曲 ID
score	DOUBLE	—	集成推荐得分
source_type	VARCHAR(20)	DEFAULT 'deepfm'	推荐来源标记
create_time	DATETIME	DEFAULT NOW()	推荐生成时间

表 4-7 推荐反馈表结构

字段名	数据类型	约束	说明
id	INT	PK,AUTO_INCREMENT	主键
user_id	INT	FK→users.id NOT NULL	用户 ID
song_id	INT	FK→songs.id NOT NULL	歌曲 ID
recommend_date	DATE	NOT NULL	推荐日期
was_played	TINYINT(1)	DEFAULT 0	是否已播

表 4-7(续表) 推荐反馈表结构

字段名	数据类型	约束	说明
play_completion	FLOAT	—	播放完成度 (0~1)
was_favorited	TINYINT(1)	DEFAULT 0	是否已收藏 (0/1)
consecutive_ignore_days	INT	DEFAULT 0	连续忽略天 数
feedback_score	FLOAT	DEFAULT 0	综合反馈得 分
cooldown_until	DATE	—	推荐冷却期 截止日
created_at	TIMESTAMP	DEFAULT NOW()	记录创建时 间
updated_at	TIMESTAMP	DEFAULT NOW()	最后更新时 间

表 4-8 用户口味反馈表结构

字段名	数据类型	约束	说明
id	INT	PK,AUTO_INCREMENT	主键
user_id	INT	FK→users.id NOT NULL	用户 ID
feedback_date	DATE	NOT NULL	反馈日期
satisfaction	ENUM	NOT NULL	用户满意度
genres_added	VARCHAR(255)	—	新增偏好流派
artists_added	VARCHAR(255)	—	新增偏好艺术家
created_at	TIMESTAMP	DEFAULT NOW()	记录创建时间

表 4-9 申诉表结构

字段名	数据类型	约束	说明
id	INT	PK,AUTO_INCREMENT	主键
username	VARCHAR(50)	NOT NULL	申诉账号名
user_id	INT	FK→users.id	关联用户 ID
appeal_type	ENUM	NOT NULL	frozen/deleted
reason	TEXT	—	申诉理由
contact_email	VARCHAR(100)	—	联系邮箱
status	ENUM	DEFAULT 'pending'	pending/approved/rejected
admin_reply	TEXT	—	管理员回复内容
create_time	TIMESTAMP	DEFAULT NOW()	申诉创建时间
update_time	TIMESTAMP	DEFAULT NOW()	最后处理时间

表 4-10 用户屏蔽内容表结构

字段名	数据类型	约束	说明
id	INT	PK,AUTO_INCREMENT	主键
user_id	INT	FK→users.id NOT NULL	用户 ID
block_type	ENUM	NOT NULL	屏蔽类型
block_value	VARCHAR(100)	NOT NULL	屏蔽的流派名或歌手名
block_count	INT	DEFAULT 1	屏蔽次数
blocked_at	TIMESTAMP	DEFAULT NOW()	最近一次屏蔽时间
blocked_until	DATE	NOT NULL	屏蔽到期日
is_active	TINYINT(1)	DEFAULT 1	是否生效（1/0）

表 4-11 歌曲滚动统计表结构

字段名	数据类型	约束	说明
song_id	INT	PK,FK→songs.id,NOTNULL	歌曲 ID（主键，与歌曲主表一对一）
cnt_7d	INT	NOT NULL,DEFAULT 0	近 7 天播放次数
cnt_30d	INT	NOT NULL,DEFAULT 0	近 30 天播放次数
trending	FLOAT	NOT NULL,DEFAULT 0	热度趋势（7 天播放量/30 天日均+1）
total_plays	BIGINT	NOT NULL,DEFAULT 0	累计总播放次数
updated_at	TIMESTAMP	NOT NULL,DEFAULT NOW() ON UPDATE NOW()	最后更新时间